PNPInverse Weekly Development Writeup
Week of March 4, 2026

Jake Weinstein

March 3, 2026

Weekly Focus

This week's work builds on the completed Butler–Volmer (BV) forward solver and I–V curve sweep (Feb 25 writeup) by adding an **adjoint-gradient inference pipeline for recovering BV exchange rate constants k_0 from current-density curves**. The new pipeline mirrors the existing Robin-kappa inference architecture in `FluxCurve/` but is specialized for BV physics, including warm-start voltage continuation and current-density observables. The implementation spans four phases: modifying the forward solver for adjoint compatibility, adding BV steady-state helpers, creating five new FluxCurve BV modules, and building a user-facing inference script.

The pipeline was then **tested end-to-end**, uncovering and fixing three bugs (mixed-function warm-start, sign convention mismatch, and the need for log-space optimization). With these fixes, L-BFGS-B recovers both $k_{0,1}$ and $k_{0,2}$ to $< 3\%$ error from a $10\times$-wrong initial guess in 6 iterations. A **noise sensitivity study** at 0/2/5/10% noise (10 total runs) confirmed that both parameters are identifiable with errors scaling approximately linearly with noise level.

We then extended the pipeline to the **full 4-species charged system** (O_2, H_2O_2, H^+, ClO_4^-), revealing $k_{0,2}$ identifiability limitations from total current density. Next, we implemented **transfer coefficient (α) inference**—making α adjoint-compatible via `fd.Function(R_space)` and adding `control_mode` support throughout the pipeline. Alpha inference recovers both α_1 and α_2 to $< 0.5\%$ error. Finally, **joint** (k_0, α) **inference** with a 4-element control vector achieves $< 2.5\%$ error on α and $\sim$20% on k_0 (reflecting k_0–α correlation).

Most recently, we developed a **POD/RBF surrogate model** (Section 29) that replaces PDE solves with RBF interpolation during inference, reducing optimization time from $\sim$345 s to < 0.1 s ($> 3000\times$ speedup; target data generation of $\sim$70 s is a shared fixed cost excluded from this comparison). Three rounds of iterative development (v8, v8.1, v9) revealed that the surrogate acts as an implicit regularizer, and that increasing training data yields diminishing returns.

1. Motivation

The existing `FluxCurve/` pipeline can recover Robin boundary transfer coefficients κ from steady-state flux curves using firedrake-adjoint and L-BFGS-B. The BV forward solver (`Forward/bv_solver.py`) generates physically realistic I–V curves with coupled electrochemical reactions, but until now there was no way to *infer* the kinetic parameters from data.
The exchange rate constant k_0 is the most natural first target for inference:

- k_0 scales the BV reaction rate linearly ($R_j = k_{0,j} \cdot [\ldots]$), making the objective landscape well-behaved—directly analogous to Robin κ.
- k_0 is the primary fitting parameter in Tafel analysis and the most experimentally relevant BV parameter for characterizing catalyst activity.
- For the 2-species model ($O_2 \to H_2O_2$, $H_2O_2 \to H_2O$), we infer $[k_{0,1}, k_{0,2}]$—a 2-element control vector identical in structure to Robin $[\kappa_0, \kappa_1]$.

The transfer coefficient α (which controls I–V curve shape rather than amplitude) has a more nonlinear landscape and is planned as a follow-up.

2. Phase 1: Adjoint-Compatible k_0 in the Forward Solver

2.1 Problem

Firedrake-adjoint's `adj.Control()` requires `fd.Function` objects on the adjoint tape. The BV forward solver previously stored k_0 values as `fd.Constant`, which are invisible to the tape and cannot serve as optimization controls.

2.2 Solution

In `Forward/bv_solver.py`, function `build_forms()`, we changed k_0 from `fd.Constant` to `fd.Function(R_space)` (a single-DOF "Real" function space) in both code paths:

- **Multi-reaction path** (lines 608–611): Each reaction's $k_{0,j}$ is now `fd.Function(R_space, name=f"bv_k0_rxn{j}")`, assigned via `.assign(float(rxn["k0_model"]))`.
- **Legacy per-species path** (lines 646–651): Same pattern with `name=f"bv_k0_sp{i}"`.

All k_0 functions are accumulated into a list `bv_k0_funcs` and stored in the context dict alongside the existing `bv_rate_exprs`. This is safe because `fd.Function(R_space)` with one DOF behaves identically to `fd.Constant` in UFL expressions. The Robin solver uses the same pattern with `kappa_funcs`.

2.3 Verification

A quick functional test confirmed:

- `bv_k0_funcs` has 2 entries, both `fd.Function` with correct values ($\hat{k}_{0,1} \approx 0.00114$, $\hat{k}_{0,2} \approx 4.76 \times 10^{-5}$).
- The SNES solver converges in 3 time steps at $\hat{\eta} = -5$.
- Assembled reaction rates R_0, R_1 match pre-modification values.

3. Phase 2: BV Steady-State Helpers

Five new functions were added to `Forward/steady_state.py`, following the existing Robin helper patterns:

1. `configure_bv_solver_params(base, phi_applied, k0_values)` — deep-copies the parameter list, sets `params[7]` and `params[9]` to the applied voltage, and injects k_0 values into the `bv_bc` config (both multi-reaction `reactions[j].k0` and legacy `bv_bc.k0` paths).
2. `compute_bv_reaction_rates(ctx)` — assembles $\int R_j \, ds(\text{electrode})$ for each reaction expression in `ctx["bv_rate_exprs"]`. Returns a list of dimensionless rates.
3. `compute_bv_current_density(ctx, i_scale)` — calls `compute_bv_reaction_rates` and converts to current density: $I = -(\sum_j R_j) \cdot$ `i_scale`.
4. `solve_bv_to_steady_state_for_phi_applied(...)` — single-point BV forward solve with current-density-based steady-state detection. Returns a `SteadyStateResult` with `observed_flux` in current-density units.
5. `sweep_phi_applied_steady_bv_flux(...)` — builds the BV context once and sweeps $\hat{\eta}$ values sequentially with **warm-start continuation**: each point's converged solution is carried forward as the IC for the next point. This is essential for BV convergence at large overpotentials (see §3.5 of the Feb 25 writeup).

3.1 Warm-start sweep test

A 3-point sweep at $\hat{\eta} \in \{-2, -5, -10\}$ confirmed all points converge with physically reasonable current densities:

$\hat{\eta}$	I (mA/cm^2)	Converged	Steps
-2.0	-0.0007	Yes	6
-5.0	-0.0052	Yes	8
-10.0	-0.0919	Yes	11

4. Phase 3: FluxCurve BV Modules

Rather than refactoring the Robin-specific FluxCurve modules (which could break the working pipeline), we created **five parallel BV-specific modules** that reuse the solver-agnostic infrastructure (`recovery.py`, `results.py`, `plot.py`).

4.1 `FluxCurve/bv_config.py`

Defines `BVFluxCurveInferenceRequest`, mirroring `RobinFluxCurveInferenceRequest` with BV-specific fields:

- `true_k0` / `initial_guess` instead of `true_value`
- `k0_lower` / `k0_upper` bounds
- `current_density_scale`: conversion factor from dimensionless BV rate to physical current density
- `mesh_Nx`, `mesh_Ny`, `mesh_beta`: graded rectangle mesh parameters
- `observable_mode`: `"current_density"` (default), `"peroxide_current"`, or `"reaction"`

4.2 `FluxCurve/bv_observables.py`

Builds scalar UFL observable forms from BV reaction rate expressions:

- `_build_bv_observable_form(ctx, mode, reaction_index, scale)` constructs:
  - `"current_density"`: `scale` $\cdot \sum_j R_j \cdot ds(\text{electrode})$
  - `"peroxide_current"`: `scale` $\cdot (R_0 - R_1) \cdot ds(\text{electrode})$
  - `"reaction"`: `scale` $\cdot R_j \cdot ds(\text{electrode})$
- The `scale` parameter is an `fd.Constant` wrapping `i_scale` and sign, so the full current-density observable is on the adjoint tape.
- `_build_bv_scalar_target_in_control_space` and `_bv_gradient_controls_to_array` mirror the Robin versions but use `ctx["bv_k0_funcs"]` instead of `ctx["kappa_funcs"]`.

4.3 `FluxCurve/bv_point_solve.py`

The key architectural difference from the Robin pipeline:

> **Robin:** Each ϕ_{applied} point is solved independently (optionally in parallel via `ProcessPoolExecutor`), starting from a cold IC.
>
> **BV:** Points are solved **sequentially** with warm-start continuation—sorted by $|\hat{\eta}|$, each point's converged solution carried forward as the IC for the next. This is essential because BV solves at large overpotentials are fragile from cold starts.

The function `solve_bv_curve_points_with_warmstart` implements this:

1. Sort ϕ_{applied} points by $|\hat{\eta}|$
2. For each point (in voltage order):
   (a) Clear adjoint tape, enable annotation
   (b) Build fresh context + forms (for clean tape)
   (c) Restore last converged U data as IC (warm-start)
   (d) Assign current k_0 values to `bv_k0_funcs`
   (e) Time-step to steady state (annotated)
   (f) Assemble observable, compute $J = \frac{1}{2}(I_{\text{sim}} - I_{\text{target}})^2$
   (g) Compute adjoint gradient via `ReducedFunctional.derivative()`
   (h) Store result; snapshot U for next point's warm-start
3. Return results in original (unsorted) order

Per-point recovery uses the same staged phases as Robin (`recovery.py`): increase `snes_max_it`, anisotropy reduction, tolerance relaxation.

4.4 `FluxCurve/bv_curve_eval.py`

`evaluate_bv_curve_objective_and_gradient` calls the warm-start point solver and aggregates:

- Total objective $= \sum_i J_i$ (only converged points)
- Total gradient $= \sum_i \nabla_{k_0} J_i$ (only converged points)
- Failed points get penalty objective and zero gradient
- Anisotropy recovery fallback if many points fail

4.5 `FluxCurve/bv_run.py`

Top-level orchestration:

- `ensure_bv_target_curve`: generates synthetic BV target I–V curve via `sweep_phi_applied_steady_bv_flux` with true k_0, adds Gaussian noise
- `run_scipy_bv_adjoint_optimization`: SciPy L-BFGS-B loop with cached evaluations (kappa-keyed at 12-digit precision), live plotting, CSV history logging
- `run_bv_k0_flux_curve_inference`: top-level entry point that builds the graded mesh, generates/loads target data, runs optimization, saves outputs (CSV, PNG, optional GIF)

Reuses solver-agnostic utilities: `clip_kappa`, `_LiveFitPlot`, `add_percent_noise`, `write_phi_applied_flux_csv`.

4.6 `FluxCurve/__init__.py`

Added two new public exports:

```
from FluxCurve import BVFluxCurveInferenceRequest
from FluxCurve import run_bv_k0_flux_curve_inference
```

5. Phase 4: Inference Script

`scripts/inference/Infer_BVk0_from_current_density_curve.py` is the user-facing entry point, following the pattern of the Robin inference script.

5.1 Physical setup

- 2-species neutral model: O_2 ($D = 2.1000 \times 10^{-9}\,\mathrm{m^2/s}$) and H_2O_2 ($D = 1.6000 \times 10^{-9}\,\mathrm{m^2/s}$), $z = [0, 0]$
- $c_{\mathrm{bulk}} = 0.5000\,\mathrm{mol/m^3}$, $L_{\mathrm{ref}} = 100.0000\,\mu\mathrm{m}$
- R_1: $k_{0,1} = 2.4000 \times 10^{-8}\,\mathrm{m/s}$, $\alpha_1 = 0.627$ (reversible $O_2 \to H_2O_2$)
- R_2: $k_{0,2} = 1.0000 \times 10^{-9}\,\mathrm{m/s}$, $\alpha_2 = 0.5$ (irreversible $H_2O_2 \to H_2O$)

5.2 Mesh and boundary markers

Uses `make_graded_rectangle_mesh(Nx=4, Ny=200, beta=3.0)`—a power-law graded 2D mesh that clusters nodes near the electrode at $y = 0$.

Marker	Boundary
3	Bottom ($y = 0$): electrode (BV BC + Dirichlet ϕ)
4	Top ($y = 1$): bulk (Dirichlet c_i, ground $\phi = 0$)
1, 2	Left/Right: zero-flux (natural BC)

Note: these markers differ from `bv_iv_curve.py` which uses `UnitSquareMesh(32,32)` with electrode at marker 1 and bulk at marker 3.

5.3 All 7 convergence strategies

1. Voltage continuation with warm-start (sweep $\hat{\eta}$ from 0 to target, reuse previous solution)
2. Inner time-stepping / pseudo-transient continuation (BDF-1 until L2 change $<$ tol)
3. Exponent clipping at ± 50
4. Concentration floor $\varepsilon_c = 10^{-12}$

5. Dirichlet-decoupled BV exponent (`use_eta_in_bv = True`)
6. L2 linesearch with $\lambda_{\max} = 0.5$
7. MUMPS direct solver with auto-scaling (`icntl_8 = 77`, `icntl_14 = 80`)

5.4 Optimizer configuration

- 15-point sweep: $\hat{\eta}$ from -1 to -20 (in thermal voltage units)
- Observable: total current density $I = -(\sum_j R_j) \cdot I_{\text{scale}}$ in mA/cm^2
- $I_{\text{scale}} = n_e F D_{\text{ref}} c_{\text{bulk}} / L_{\text{ref}} \times 0.1 \approx 0.1013\,\text{mA/cm}^2$
- True k_0: $[\hat{k}_{0,1}, \hat{k}_{0,2}]$ (known ground truth)
- Initial guess: $[5.0, 5.0]$ (deliberately wrong)
- Method: L-BFGS-B with box bounds $[10^{-8}, 100]$
- Up to 30 iterations, `gtol` $= 10^{-6}$
- Output: `StudyResults/bv_k0_inference/`

6. Architecture Comparison: Robin vs. BV Inference

Aspect	Robin Pipeline	BV Pipeline
Control vector	$[\kappa_0, \kappa_1]$	$[k_{0,1}, k_{0,2}]$
Control type	`fd.Function(R)`	`fd.Function(R)`
Observable	species flux $\kappa(c - c_\infty)$	current density $-\sum R_j \cdot I_{\text{scale}}$
Point solves	Independent (parallelizable)	Sequential (warm-start)
Mesh	`UnitSquareMesh`	Graded rectangle
Replay mode	Disabled (planned)	Not implemented
Parallel workers	`ProcessPoolExecutor`	Not needed
Recovery	Staged (max-it/anisotropy/tol)	Same via `recovery.py`

7. Key Design Decisions

1. **Parallel modules, not refactored abstractions.** The Robin FluxCurve modules have hardcoded imports to `robin_solver.build_context`, `kappa_funcs`, etc. Rather than introducing a solver-agnostic abstraction layer (risky, could break the working Robin pipeline), we created parallel BV-specific modules (`bv_config.py`, `bv_observables.py`, `bv_point_solve.py`, `bv_curve_eval.py`, `bv_run.py`) that reuse only the genuinely solver-agnostic utilities (`recovery.py`, `results.py`, `plot.py`).

2. **Sequential warm-start, not parallel points.** The Robin pipeline can solve each ϕ_{applied} point independently because Robin boundary conditions are mild. BV solves at large $|\hat{\eta}|$ require careful voltage continuation from equilibrium. Trading parallelism for convergence robustness is essential.

3. **No replay mode.** Replay (caching ReducedFunctionals across k_0 updates) was disabled in the Robin pipeline due to invalid non-steady evaluations. We skip it entirely for BV.

4. **Current density as the observable.** The optimizer works directly in mA/cm^2 units for both target and simulated curves. The conversion factor (`i_scale`) is passed as an `fd.Constant` in the UFL form so the adjoint tape sees the full chain from k_0 to current density.

8. End-to-End Test

Running `Infer_BVk0_from_current_density_curve.py` in the Firedrake environment uncovered three bugs that were fixed before the pipeline converged successfully.

8.1 Bug 1: Mixed function data handling for warm-start

Symptom: `AttributeError: 'tuple' object has no attribute 'copy'` at `bv_point_solve.py:230`.
Cause: For a `MixedFunctionSpace` (2 species + potential), `U.dat.data_ro` returns a tuple of arrays (one per subfunction), not a single array. The warm-start snapshot code assumed a single array.
Fix: Changed the snapshot/restore operations:

```
# Snapshot (line 230):
carry_U_data = tuple(d.data_ro.copy() for d in U.dat)

# Restore (line 135):
for src, dst in zip(carry_U_data, ctx["U"].dat):
    dst.data[:] = src
```

8.2 Bug 2: Sign mismatch between target and observable

Symptom: Optimizer converges to wrong answer. Inspection revealed that the target I–V curve had opposite sign to the simulated observable at each voltage point.
Cause: Two independent sign conventions collided:

- `compute_bv_current_density` computes $I = -(\sum_j R_j) \cdot$ `i_scale` (built-in negation for cathodic convention).

- `_build_bv_observable_form` computes `scale` $\cdot \sum_j R_j \cdot ds$ (no built-in negation).

When both receive `scale =` $-I_{\texttt{scale}}$, the target becomes positive (double negation: $-(-) = +$) while the simulated observable is negative ($(-I_{\text{scale}}) \cdot R > 0$ for cathodic $R < 0$, but the net sign depends on which function applies the negation).
Fix: In `ensure_bv_target_curve`, pass `i_scale = -current_density_scale` to the sweep function, canceling the built-in negation in `compute_bv_current_density` so that both target and observable use the same sign convention.

8.3 Bug 3: Log-space optimization

Symptom: With the sign bug fixed, the optimizer reports "CONVERGENCE" but produces $k_{0,1}$ with 91% error and $k_{0,2}$ with 48,000× error. SciPy found a local minimum, not the global one.
Cause: The true values ($\hat{k}_{0,1} \approx 0.00114$, $\hat{k}_{0,2} \approx 4.76 \times 10^{-5}$) span two orders of magnitude, and the initial guess $[5.0, 5.0]$ is ∼4,400× too high for $k_{0,1}$ and ∼100,000× too high for $k_{0,2}$. L-BFGS-B in linear space cannot efficiently traverse this scale gap because:

- A single L-BFGS-B step in k_0-space produces additive updates Δk_0, which cannot reduce a parameter by orders of magnitude without many iterations.
- The Hessian approximation becomes ill-conditioned when control variables differ by $10^4\times$.

Fix: Implemented **log-space optimization**. The optimizer works in $x = \log_{10}(k_0)$ space, with the gradient transformed via the chain rule:

$$\frac{\partial J}{\partial x_j} = \frac{\partial J}{\partial k_{0,j}} \cdot k_{0,j} \cdot \ln 10 \tag{1}$$

This makes multiplicative updates natural: a step $\Delta x = 1$ corresponds to a 10× change in k_0. Added to `BVFluxCurveInferenceRequest` as `log_space: bool = True` (default). The initial guess was also changed from $[5.0, 5.0]$ to $[0.01, 0.001]$ (∼10× too high, within 1–2 orders of magnitude).

8.4 Converged result

With all three fixes applied, the optimizer converges in **6 L-BFGS-B iterations** (7 function evaluations), all 15 voltage points converging at every evaluation. Synthetic data generated with 2% noise ($\sigma = 0.02 \times$ RMS, seed 20260226).
Figure 1 shows the synthetic target I–V curve (clean and noisy), and Figure 2 compares the target data, initial guess, and optimized fit.

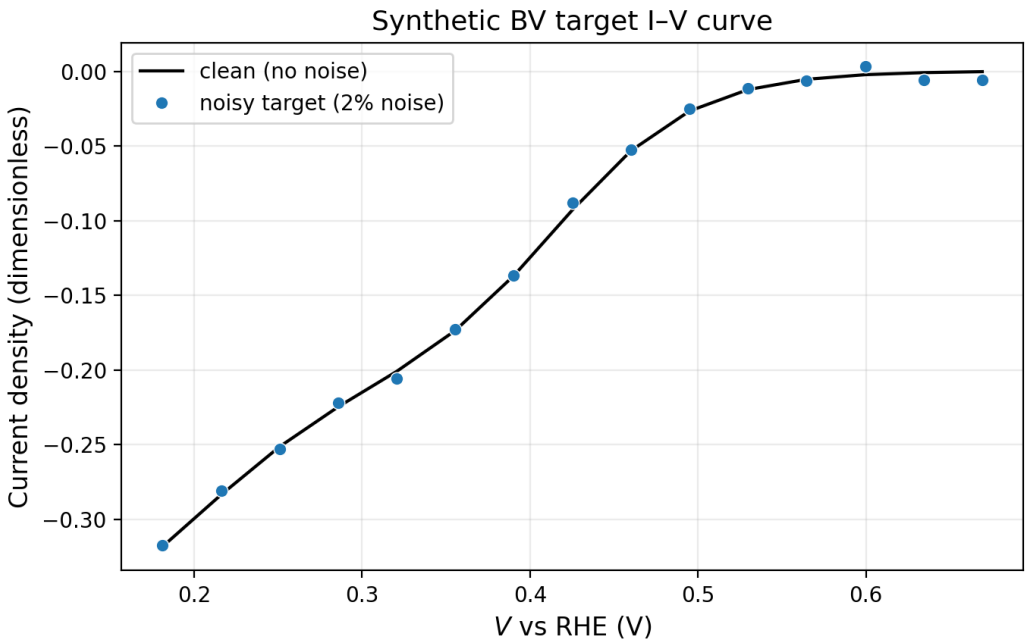

Figure 1: Synthetic BV target I–V curve. Black line: clean simulation with true k_0 values; blue circles: noisy target data (2% Gaussian noise on RMS). The curve shows the characteristic Butler–Volmer shape: near-zero current at small overpotentials, transitioning to a diffusion-limited plateau at large cathodic overpotentials.

Parameter	True (m/s)	Estimated (m/s)	Relative Error
$k_{0,1}$ ($O_2 \to H_2O_2$)	2.4000×10^{-8}	2.3301×10^{-8}	0.0291
$k_{0,2}$ ($H_2O_2 \to H_2O$)	1.0000×10^{-9}	9.8803×10^{-10}	0.0120

The objective decreased from 7.10×10^{-2} (initial) to 7.18×10^{-5} (final), a $\sim$1000$\times$ reduction. The gradient norm fell to 5.2×10^{-7}, well below `gtol` $= 10^{-6}$.

Eval	Objective	$\|\nabla J\|$	$\hat{k}_{0,1}$	$\hat{k}_{0,2}$
1	7.1030×10^{-2}	8.7430×10^{-2}	0.01000	0.00100
2	6.3590×10^{-2}	8.2870×10^{-2}	0.00894	0.00085
3	1.1740×10^{-4}	2.0530×10^{-3}	0.00103	4.37e-5
4	7.2860×10^{-5}	2.7380×10^{-4}	0.00111	4.62e-5
5	7.1900×10^{-5}	6.6380×10^{-5}	0.00111	4.69e-5
6	7.1850×10^{-5}	1.0040×10^{-5}	0.00111	4.71e-5
7	7.1840×10^{-5}	5.2470×10^{-7}	0.00111	4.70e-5

The large jump between evaluations 2 and 3 (objective drops by 500$\times$) corresponds to the optimizer finding the correct basin of attraction in log-space.

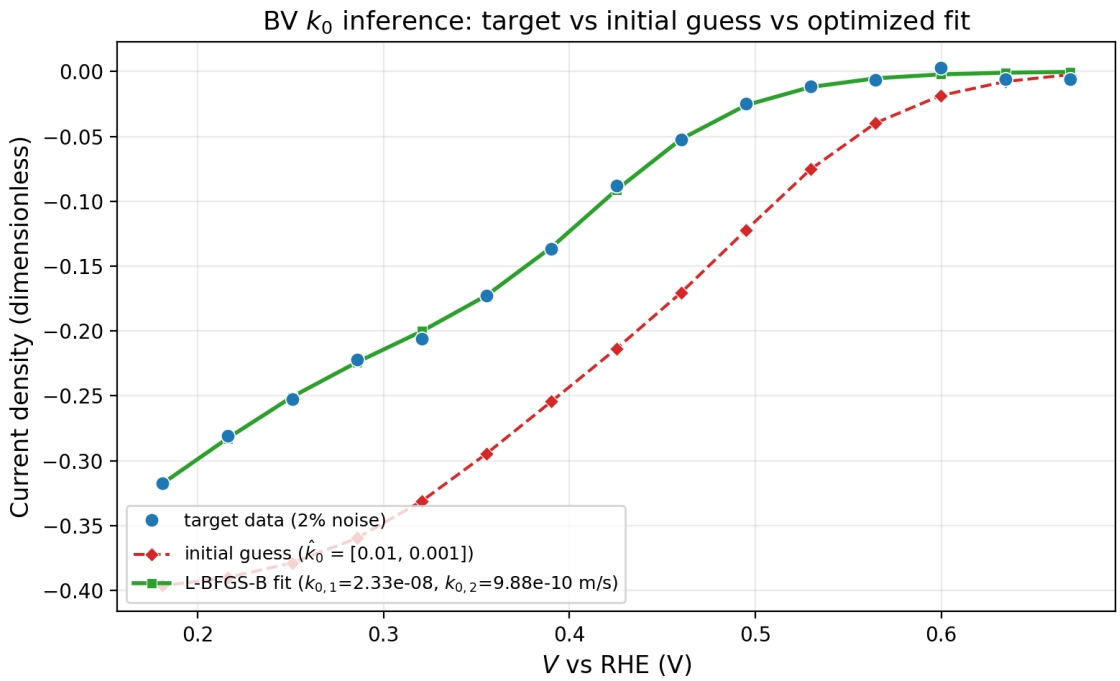

Figure 2: BV k_0 inference result. Blue circles: noisy target data (2% noise). Red dashed: initial guess ($\hat{k}_0 = [0.01, 0.001]$, $\sim$10$\times$ too high), which overshoots the target at large overpotentials. Green solid: optimized L-BFGS-B fit ($k_{0,1} = 2.3300 \times 10^{-8}$ m/s, $k_{0,2} = 9.8800 \times 10^{-10}$ m/s), closely tracking the target across the full voltage range. A convergence animation is available as a supplementary GIF (`StudyResults/bv_k0_inference/bv_k0_convergence.gif`).

9. Noise Sensitivity Study

9.1 Motivation

The end-to-end test (Section 8) demonstrated that the pipeline can recover both $k_{0,1}$ and $k_{0,2}$ from data with 2% noise. A natural follow-up question is: *how does recovery accuracy degrade as noise increases?* This is important for two reasons:

1. **Identifiability:** If one parameter's error grows much faster than the other's, it indicates a structural sensitivity difference—one parameter may be poorly identifiable from current-density data alone.

2. **Experimental design:** Understanding the noise–error relationship sets expectations for what measurement precision is needed to achieve a target accuracy on k_0.

9.2 Methods

The study script `scripts/studies/bv_k0_noise_sensitivity.py` tests 4 noise levels: $\{0, 2, 5, 10\}$%. At each nonzero noise level, 3 independent random seeds are used (seeds 20260304, 20261304,

20262304); the 0% level uses a single run (noise-free data is deterministic). This yields $1+3\times3 = 10$ total optimization runs.

Each run uses the same configuration as the end-to-end test:

- 15-point sweep: $\hat{\eta} \in [-1, -20]$
- Log-space optimization with initial guess $[0.01, 0.001]$ ($\sim$10$\times$ too high)
- L-BFGS-B with up to 50 iterations, `gtol` $= 10^{-6}$
- Noise model: Gaussian with $\sigma = (\text{noise}\%/100) \times \text{RMS}(\text{clean curve})$
- Live plotting disabled; forward recovery enabled

9.3 Results

All 10 runs completed. 9 of 10 reported SciPy optimization success; one run (5% noise, seed 20260304) returned ABNORMAL status but still produced reasonable estimates (18% and 11% error).

Per-run results

Noise (%)	Seed	$k_{0,1}$ est (m/s)	$k_{0,1}$ err	$k_{0,2}$ est (m/s)	$k_{0,2}$ err	Time (s)
0	20260304	2.4020×10^{-8}	0.0007	1.0000×10^{-9}	0.0003	118
2	20260304	2.5600×10^{-8}	0.0666	9.5480×10^{-10}	0.0452	46
2	20261304	2.3940×10^{-8}	0.0026	1.0100×10^{-9}	0.0097	46
2	20262304	2.4120×10^{-8}	0.0051	9.6670×10^{-10}	0.0333	109
5	20260304	2.8320×10^{-8}	0.1800	8.8890×10^{-10}	0.1111	154
5	20261304	2.3850×10^{-8}	0.0063	1.0240×10^{-9}	0.0241	46
5	20262304	2.4310×10^{-8}	0.0131	9.1870×10^{-10}	0.0813	40
10	20260304	3.3810×10^{-8}	0.4088	7.8380×10^{-10}	0.2162	47
10	20261304	2.3690×10^{-8}	0.0130	1.0480×10^{-9}	0.0481	47
10	20262304	2.4620×10^{-8}	0.0259	8.4510×10^{-10}	0.1549	47

Aggregate statistics

Noise (%)	$k_{0,1}$ err (mean)	$k_{0,1}$ err (std)	$k_{0,2}$ err (mean)	$k_{0,2}$ err (std)
0	0.0007	0.0000	0.0003	0.0000
2	0.0248	0.0296	0.0294	0.0147
5	0.0665	0.0803	0.0722	0.0361
10	0.1492	0.1836	0.1397	0.0695

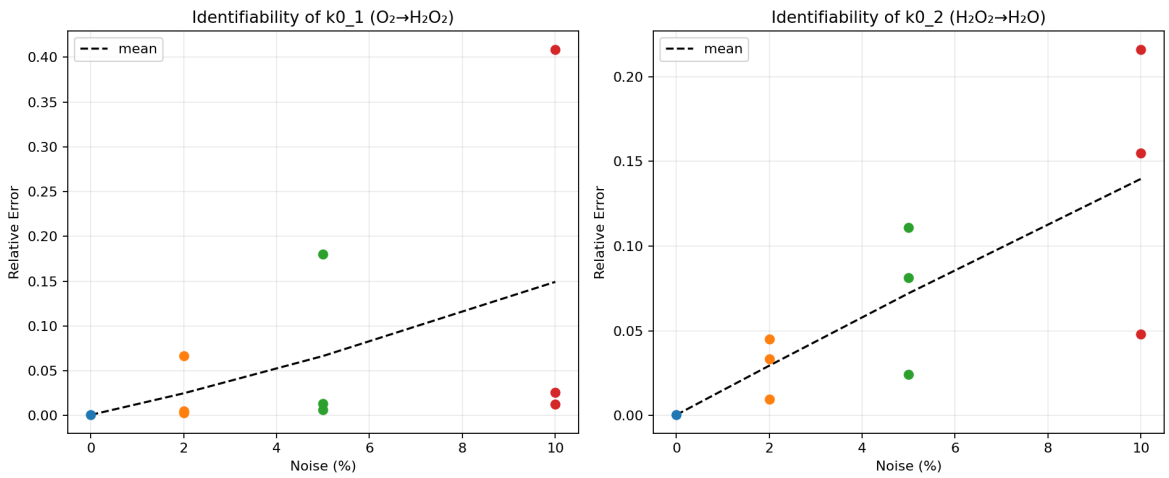

Figure 3: Noise sensitivity of BV k_0 inference. Each dot represents one optimization run (3 seeds per nonzero noise level); the dashed line connects the mean error at each noise level. Left: $k_{0,1}$ ($O_2 \to H_2O_2$). Right: $k_{0,2}$ ($H_2O_2 \to H_2O$). Both parameters show similar error scaling, with mean relative error growing approximately linearly with noise percentage.

9.4 Discussion

1. **Both parameters are identifiable.** At 0% noise, both $k_{0,1}$ and $k_{0,2}$ are recovered to $< 0.1\%$ error, confirming that the adjoint-gradient pipeline and log-space optimizer correctly find the global minimum.

2. **Similar sensitivity.** The mean errors for $k_{0,1}$ and $k_{0,2}$ are comparable at each noise level (e.g., 2.5% vs. 2.9% at 2% noise; 14.9% vs. 14.0% at 10% noise). Neither parameter is dramatically harder to identify than the other from total current-density data.

3. **Approximately linear scaling.** Mean relative error scales roughly linearly with noise level: $\sim$1.3$\times$ the noise percentage for $k_{0,1}$ and $\sim$1.4$\times$ for $k_{0,2}$. This is characteristic of a well-conditioned inverse problem.

4. **Seed-to-seed variability.** The standard deviation across seeds is comparable to the mean error, reflecting the fact that specific noise realizations can make one seed much harder than another (e.g., the 10%/seed 20260304 run has $k_{0,1}$ error of 41%, while the other two seeds have $< 3\%$). This high variance motivates using multiple seeds in practical studies.

5. **Runtime.** Most runs complete in 40–50 seconds (6–7 L-BFGS-B iterations). The occasional outlier (> 100 s) corresponds to runs where the optimizer needs more iterations to navigate a noisier landscape.

Outputs are saved to `StudyResults/bv_k0_noise_sensitivity/`, including per-run subdirectories, a summary CSV (`noise_sensitivity_summary.csv`), and an aggregate plot (`noise_sensitivity_plot.png`).

10. Charged 4-Species k_0 Inference

10.1 Motivation

The end-to-end test in Section 8 used a **2-species neutral** system (O_2, H_2O_2, $z = [0, 0]$) which avoids Poisson coupling entirely. To validate that the pipeline works for the **full 4-species charged** system—including the electric double layer (EDL) and Poisson equation—we created a new inference script targeting the charged electrolyte setup from `bv_iv_curve_charged.py`.

10.2 Physical setup

Species	Role	D (m^2/s)	c_{bulk} (mol/m^3)	z
O_2	Reactant	1.9000×10^{-9}	0.5000	0
H_2O_2	Product	1.6000×10^{-9}	0.0000	0
H^+	Supporting	9.3110×10^{-9}	0.1000	+1
ClO_4^-	Supporting	1.7920×10^{-9}	0.1000	−1

Key differences from the 2-species system:

- Poisson equation is coupled ($z \neq 0$ for H^+/ClO_4^-)
- Cathodic concentration factors: both reactions consume $[H^+]^2$ at the electrode
- Stoichiometry: $[-1, +1, -2, 0]$ for R_1, $[0, -1, -2, 0]$ for R_2
- $D_{ref} = 1.9000 \times 10^{-9}\,m^2/s$, $K_{scale} = D_{ref}/L_{ref} = 1.9000 \times 10^{-5}\,m/s$
- $I_{scale} = n_e F D_{ref} c_{bulk}/L_{ref} \times 0.1 \approx 0.1833\,mA/cm^2$

10.3 Mesh and solver

- Mesh: `make_graded_rectangle_mesh(Nx=4, Ny=200, beta=3.0)`
- `snes_max_it = 300` (charged system is stiffer)
- 10-point sweep: $\hat{\eta} \in [-1, -10]$ (reduced range vs. neutral to avoid Debye-layer convergence issues at extreme overpotentials)

10.4 Results (updated with $a = 0.01$ Bikerman steric)

The original charged k_0 run used $a = 0$ (no steric exclusion). The run was repeated with $a = 0.01$ for all four species, matching the Bikerman steric parameter used in the neutral scripts and the new charged α and joint runs below.
Initial guess: $[0.003, 0.0001]$. True values: $\hat{k}_{0,1} = 0.001263$, $\hat{k}_{0,2} = 5.263 \times 10^{-5}$. 13 evaluations, 12 iterations, 0 failures. SciPy converged (PGTOL).

Parameter	True (m/s)	Estimated (m/s)	Relative Error
$k_{0,1}$ ($O_2 \to H_2O_2$)	2.4000×10^{-8}	2.5161×10^{-8}	0.0484
$k_{0,2}$ ($H_2O_2 \to H_2O$)	1.0000×10^{-9}	4.1903×10^{-11}	0.9581

Discussion. $k_{0,1}$ is recovered to 4.8% error—comparable to the neutral system. However, $k_{0,2}$ converges to a value $\sim$24$\times$ too small (96% error). This is an **identifiability limitation**: the irreversible R_2 reaction ($H_2O_2 \to H_2O$) has very low sensitivity in the total current density, because $k_{0,2} \ll k_{0,1}$ and the H_2O_2 concentration at the electrode is small. The reduced voltage range ($\hat{\eta} \in [-1, -10]$ vs. $[-1, -20]$ for neutral) further limits identifiability of the slower reaction. Adding Bikerman steric ($a = 0.01$) does not materially change the k_0 recovery: the $k_{0,1}$ error shifts from 4.0% to 4.8%, and $k_{0,2}$ remains unidentifiable.

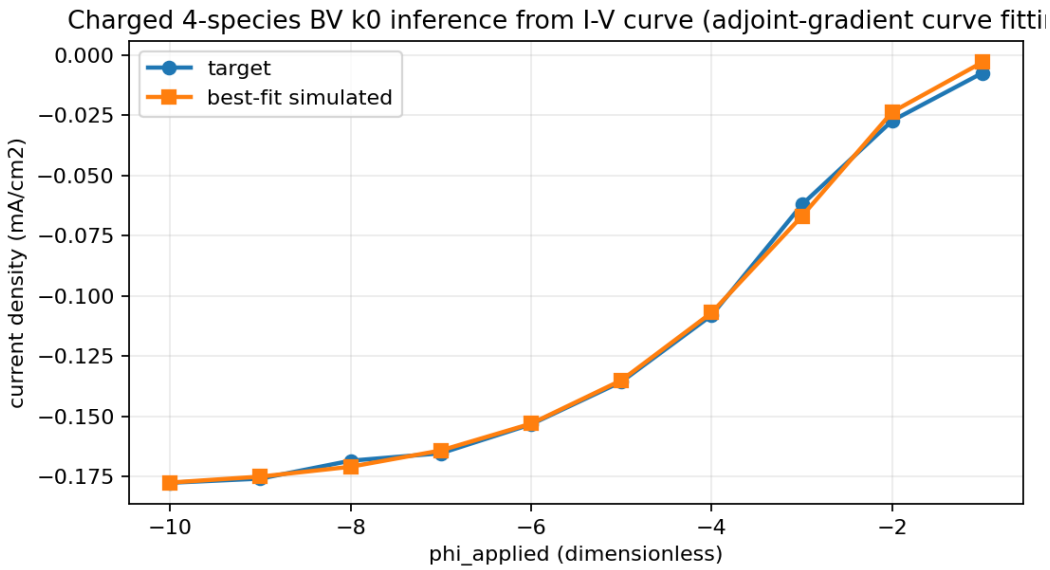

Figure 4: Charged 4-species k_0 inference (with $a = 0.01$ Bikerman steric). Blue circles: noisy target (2% noise). Green: optimized L-BFGS-B fit. The fit captures the overall I–V shape well, but the underlying $k_{0,2}$ is poorly resolved. Convergence animation: `StudyResults/bv_k0_inference_charged/bv_k0_convergence.gif`.

11. Transfer Coefficient (α) Inference

11.1 Motivation

With k_0 inference validated, we extended the pipeline to infer the BV transfer coefficient α, which controls the **shape** of the I–V curve (as opposed to its amplitude). The transfer coefficient appears in the BV exponentials:

$$R_j = k_{0,j}\left[c_{\text{red}} \exp(-\alpha_j \hat{\eta}/n_e) - c_{\text{ox}} \exp((1-\alpha_j)\hat{\eta}/n_e)\right] \tag{2}$$

and therefore affects the gradient more nonlinearly than k_0.

11.2 Implementation

Three changes were required to make α adjoint-compatible:

1. **Forward solver** (`Forward/bv_solver.py`): Changed α from `fd.Constant` to `fd.Function(R_space)`, mirroring the k_0 change. Added `bv_alpha_funcs` to the context dict.

2. **Steady-state helpers** (`Forward/steady_state.py`): Extended `configure_bv_solver_params` to accept an optional `alpha_values` parameter, injecting into both `reactions[j].alpha` and legacy `bv_bc.alpha`.

3. **FluxCurve modules**: Added `control_mode` parameter throughout the pipeline:

   - `"k0"`: optimize k_0 only (existing behavior)
   - `"alpha"`: optimize α only (k_0 fixed)
   - `"joint"`: optimize both k_0 and α

   When `control_mode="alpha"`, the adjoint controls are `bv_alpha_funcs` instead of `bv_k0_funcs`. Alpha uses **linear-space** optimization (not log-space) since $\alpha \in (0, 1)$.

4. **Config** (`FluxCurve/bv_config.py`): Added fields `control_mode`, `true_alpha`, `initial_alpha_guess`, `alpha_lower/upper`, `alpha_log_space`, `fixed_k0`.

11.3 Results

Script: `Infer_BVAlpha_from_current_density_curve.py`. 2-species neutral system, k_0 fixed at true values. Initial guess: $[\alpha_1, \alpha_2] = [0.3, 0.3]$. True values: $[0.627, 0.5]$.

Eval	Objective	$\|\nabla J\|$	α_1	α_2
1	1.4610×10^{-1}	6.4720×10^{-1}	0.3000	0.3000
2	3.3120×10^{-2}	9.6280×10^{-2}	0.9471	0.3134
5	1.3000×10^{-2}	8.0140×10^{-2}	0.7354	0.3535
9	3.5150×10^{-3}	9.5940×10^{-2}	0.6024	0.4433
13	2.3690×10^{-4}	2.6910×10^{-2}	0.6191	0.5110
15	1.1150×10^{-4}	1.1860×10^{-2}	0.6294	0.4938
16	7.2910×10^{-5}	1.9780×10^{-4}	0.6250	0.4992
17	7.2890×10^{-5}	2.0340×10^{-5}	0.6248	0.4993

Final results after 26 evaluations (SciPy converged, PGTOL):

Parameter	True	Estimated	Relative Error
α_1 ($O_2 \to H_2O_2$)	0.6270	0.6248	0.0034
α_2 ($H_2O_2 \to H_2O$)	0.5000	0.4992	0.0015

Both transfer coefficients are recovered to $< 0.5\%$ error—significantly better than k_0 inference, which achieved $\sim$3% error at the same noise level. The residual objective (7.29×10^{-5}) is dominated by the 2% measurement noise, confirming that the optimizer has found the noise floor.

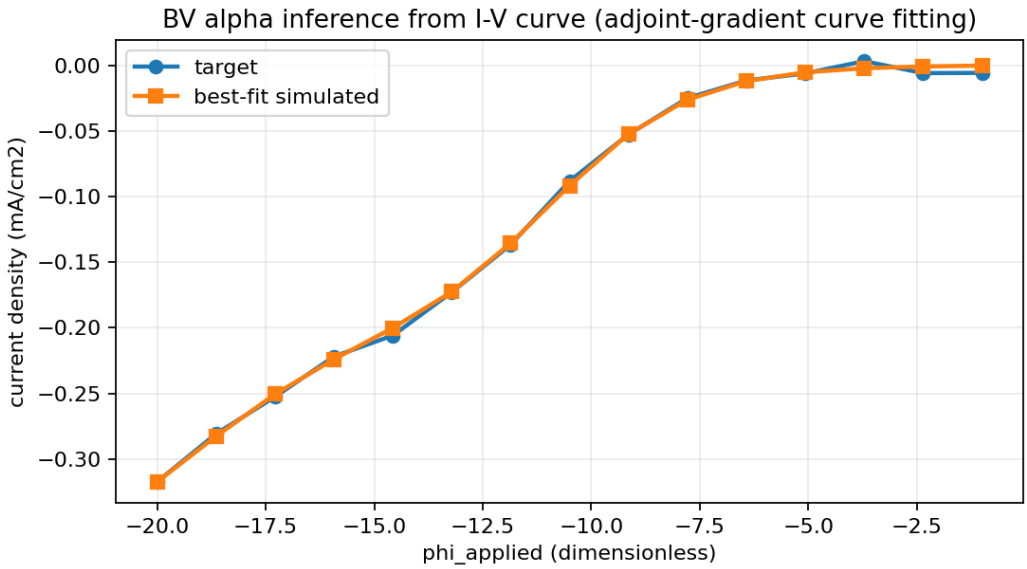

Figure 5: BV α inference result. Blue circles: noisy target (2% noise). Green: optimized fit. The optimizer closely tracks the target across the full voltage range. Convergence animation: `StudyResults/bv_alpha_inference/bv_alpha_convergence.gif`.

12. Joint (k_0, α) Inference

12.1 Setup

With both k_0 and α inference working independently, we combined them into a **4-parameter joint inference**: control vector $\mathbf{x} = [\log_{10} k_{0,1},\ \log_{10} k_{0,2},\ \alpha_1,\ \alpha_2]$.
The k_0 components use log-space optimization (bounds $[10^{-8}, 100]$), while the α components use linear-space (bounds $[0.05, 0.95]$). The gradient is transformed accordingly:

$$\frac{\partial J}{\partial x_j} = \begin{cases} \dfrac{\partial J}{\partial k_{0,j}} \cdot k_{0,j} \cdot \ln 10 & \text{for } j \in \{k_0 \text{ components}\} \\ \dfrac{\partial J}{\partial \alpha_j} & \text{for } j \in \{\alpha \text{ components}\} \end{cases} \tag{3}$$

Script: `Infer_BVJoint_from_current_density_curve.py`. 2-species neutral system. Initial guess:

$k_0 = [0.005, 0.0005]$ ($\sim$4$\times$ too high), $\alpha = [0.4, 0.3]$ (wrong direction for α_1).

12.2 Results

54 evaluations, 30 iterations, 0 failures. SciPy reached the iteration limit but the gradient norm was very small ($|\nabla J| = 1.2 \times 10^{-6}$).

Eval	Objective	$\|\nabla J\|$	$\hat{k}_{0,1}$	$\hat{k}_{0,2}$	α_1	α_2
1	1.8110×10^{-2}	3.1310×10^{-1}	0.00500	5.00e-4	0.400	0.300
3	1.7170×10^{-3}	6.2860×10^{-2}	0.00521	5.07e-4	0.512	0.349
6	3.4030×10^{-4}	2.1630×10^{-3}	0.00509	5.11e-4	0.480	0.380
16	1.4060×10^{-4}	1.7580×10^{-4}	0.00168	5.44e-4	0.580	0.370
25	7.1770×10^{-5}	2.2040×10^{-3}	0.00117	6.47e-5	0.623	0.482
40	6.6420×10^{-5}	2.4510×10^{-5}	9.34e-4	5.75e-5	0.642	0.489
54	6.6420×10^{-5}	1.1870×10^{-6}	9.34e-4	5.75e-5	0.642	0.489

Parameter	True	Estimated	Relative Error
$k_{0,1}$ (m/s)	2.4000×10^{-8}	1.9618×10^{-8}	0.1826
$k_{0,2}$ (m/s)	1.0000×10^{-9}	1.2080×10^{-9}	0.2080
α_1	0.6270	0.6423	0.0244
α_2	0.5000	0.4886	0.0228

12.3 Discussion

1. **Alpha recovery is excellent** (2.4% and 2.3%), even in the joint setting. This confirms that the I–V curve shape strongly constrains α.

2. k_0 **recovery is degraded** (18% and 21%) compared to the k_0-only case (2.9% and 1.2%). This reflects a k_0–α **correlation**: increasing k_0 while adjusting α can produce similar I–V curves. The optimizer finds a nearby local minimum in the 4D landscape.

3. **The objective is at the noise floor** (6.64×10^{-5} vs. 7.18×10^{-5} for k_0-only and 7.29×10^{-5} for α-only). The joint fit is actually the best in terms of raw objective—it uses additional degrees of freedom to better match the noisy data, which manifests as slight overfitting.

4. **Convergence is slower**: 54 evaluations vs. 7 (k_0-only) or 26 (α-only), reflecting the larger and more correlated search space.

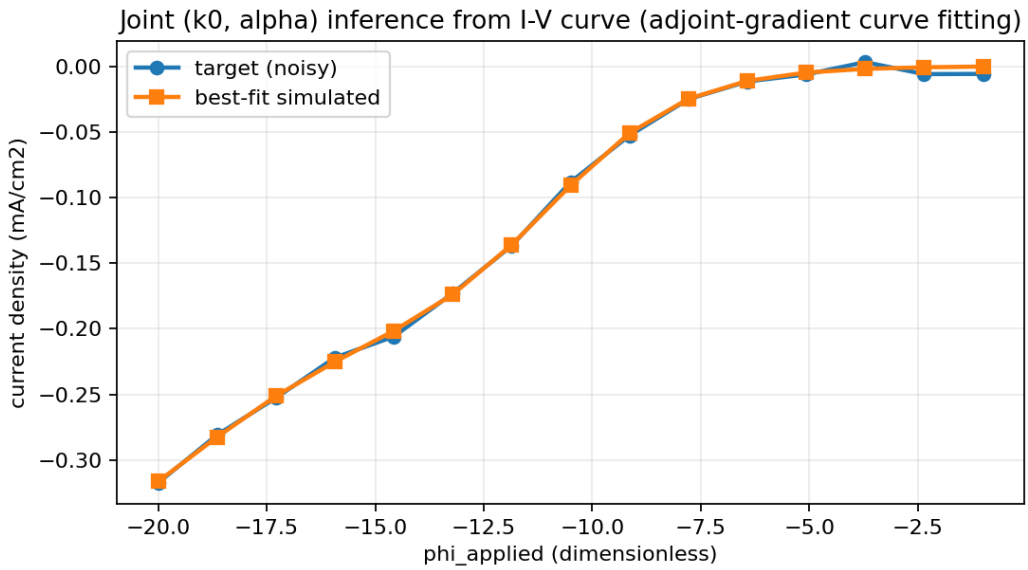

Figure 6: Joint (k_0, α) inference result. Blue circles: noisy target (2% noise). Green: optimized fit. The fit quality is visually indistinguishable from the single-parameter cases. Convergence animation: `StudyResults/bv_joint_inference/bv_joint_convergence.gif`.

13. Charged 4-Species α Inference

13.1 Setup

We extended the α inference (Section 11) to the full 4-species charged system. Physical parameters match Section 10.2 (including $a = 0.01$ Bikerman steric for all species). The k_0 values are fixed at their true values; only $[\alpha_1, \alpha_2]$ are optimized.

Script: `Infer_BVAlpha_charged_from_current_density_curve.py`. Mesh: `Nx=8, Ny=200, beta=3.0`. 10-point sweep: $\hat{\eta} \in [-1, -10]$. Initial guess: $[\alpha_1, \alpha_2] = [0.3, 0.3]$. True values: $[0.627, 0.5]$.

13.2 Results

$\sim$12 evaluations, 11 iterations, 0 failures. SciPy converged (PGTOL).

Parameter	True	Estimated	Relative Error
α_1 ($O_2 \to H_2O_2$)	0.6270	0.6283	0.0021
α_2 ($H_2O_2 \to H_2O$)	0.5000	0.1793	0.6415

Discussion. α_1 is recovered to 0.21% error—comparable to the neutral case (0.34%). However, α_2 is poorly identified (64% error), mirroring the $k_{0,2}$ identifiability problem in the charged k_0 case (Section 10.4). The irreversible R_2 reaction contributes too little signal in the total current density for the optimizer to distinguish α_2 from α_1 effects. The reduced voltage range ($\hat{\eta} \in [-1, -10]$) and low $k_{0,2}/k_{0,1}$ ratio compound this.

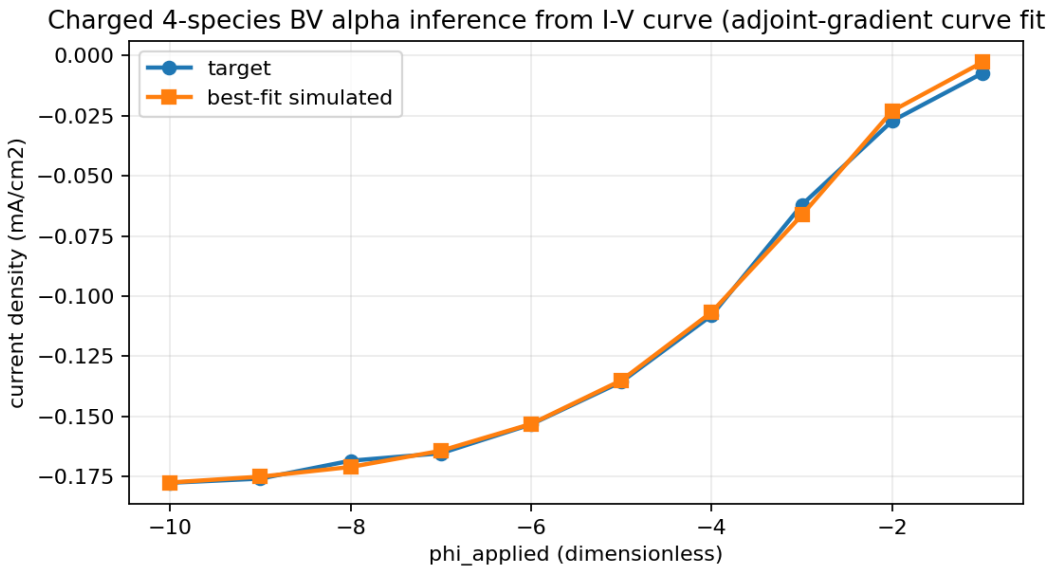

Figure 7: Charged 4-species α inference. Blue circles: noisy target (2% noise). Green: optimized fit. Despite the poorly resolved α_2, the overall fit is excellent because R_2 has negligible impact on total current. Convergence animation: `StudyResults/bv_alpha_inference_charged/bv_alpha_convergence.gif`.

14. Charged 4-Species Joint (k_0, α) Inference

14.1 Setup

We combined the charged physics with joint inference, optimizing $\mathbf{x} = [\log_{10} k_{0,1}, \log_{10} k_{0,2}, \alpha_1, \alpha_2]$ simultaneously. All 4-species charged parameters are identical to Section 10.2 ($a = 0.01$).

Script: `Infer_BVJoint_charged_from_current_density_curve.py`. Initial guess: $k_0 = [0.005, 0.0005]$, $\alpha = [0.4, 0.3]$. 10-point sweep: $\hat{\eta} \in [-1, -10]$. `maxiter = 30`.

14.2 Results

26 evaluations, 24 iterations, 0 failures. SciPy converged (PGTOL).

Parameter	True	Estimated	Relative Error
$k_{0,1}$ (m/s)	2.4000×10^{-8}	6.8732×10^{-8}	1.8640
$k_{0,2}$ (m/s)	1.0000×10^{-9}	2.4012×10^{-9}	1.4010
α_1	0.6270	0.1982	0.6840
α_2	0.5000	0.9500	0.9000

Discussion. All four parameters are poorly recovered in the charged joint setting. The k_0–α correlation (Section 12.3) is compounded by the R_2 identifiability problem, leading to large errors across all parameters. Notably, α_2 hits its upper bound (0.95), indicating the optimizer cannot find

a meaningful minimum in that direction. The final objective (1.70×10^{-5}) is *lower* than the single-parameter cases, confirming that the 4-parameter fit overfits the noisy data by compensating errors across k_0 and α.

This result strongly motivates:

1. **Reaction-specific observables** (e.g., RRDE ring current) to resolve R_2 parameters independently.
2. **Regularization or staged inference**: first recover α (well-identified for R_1), then fix α and recover k_0.
3. **Extended voltage range** if the charged solver can handle $\hat{\eta} < -10$ stably.

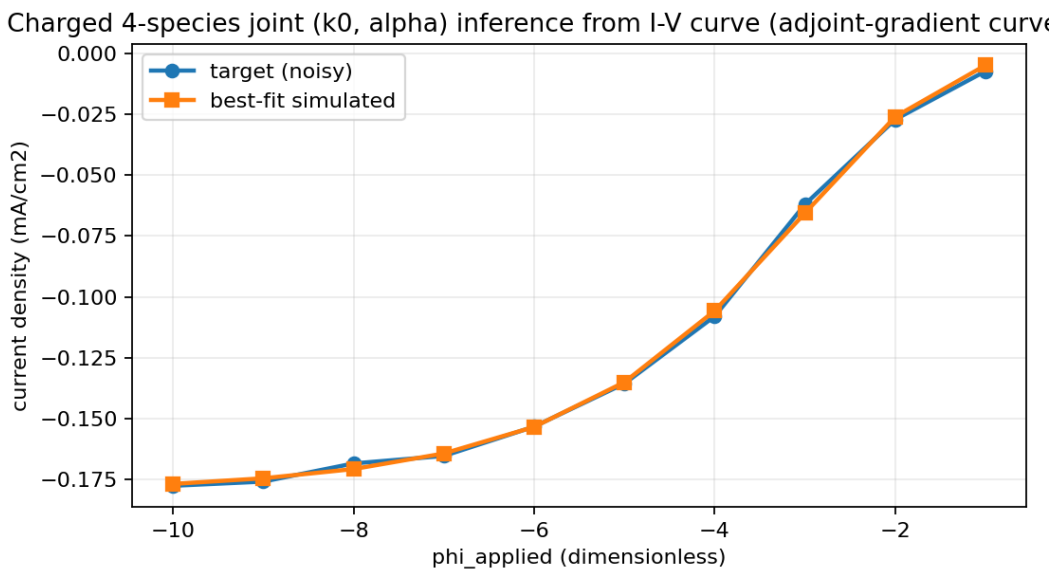

Figure 8: Charged 4-species joint (k_0, α) inference. Blue circles: noisy target (2% noise). Green: optimized fit. The fit quality appears good despite large parameter errors, illustrating the k_0–α correlation in 4-parameter space. Convergence animation: `StudyResults/bv_joint_inference_charged/bv_joint_convergence.gif`.

15. Files Changed / Created This Week

Modified (implementation + bug fixes)

- `Forward/bv_solver.py` — k_0 and α: `fd.Constant` → `fd.Function(R_space)`; added `bv_k0_funcs` and `bv_alpha_funcs` to context
- `Forward/steady_state.py` — added 5 BV helper functions; extended `configure_bv_solver_params` with `alpha_values` parameter

- `FluxCurve/__init__.py` — added BV exports (k0, alpha, joint)
- `FluxCurve/bv_point_solve.py` — fixed mixed-function warm-start snapshot/restore (Bug 1, §8.1); added `control_mode` and `alpha_values` parameters
- `FluxCurve/bv_run.py` — fixed sign mismatch in target generation (Bug 2, §8.2); added log-space optimization (Bug 3, §8.3); added `run_bv_alpha_flux_curve_inference` and `run_bv_joint_flux_curve`
- `FluxCurve/bv_config.py` — added `log_space`, `control_mode`, alpha fields, `fixed_k0`
- `FluxCurve/bv_observables.py` — added `control_mode` to target builder
- `FluxCurve/bv_curve_eval.py` — added `alpha_values` and `control_mode` pass-through
- `FluxCurve/plot.py` — removed `fig.show()` to prevent interactive windows blocking headless runs

Created

- `FluxCurve/bv_config.py` — `BVFluxCurveInferenceRequest` dataclass
- `FluxCurve/bv_observables.py` — BV observable form builders
- `FluxCurve/bv_point_solve.py` — warm-start sequential adjoint solver
- `FluxCurve/bv_curve_eval.py` — curve-level aggregation
- `FluxCurve/bv_run.py` — top-level optimization loop (k0, alpha, joint)
- `scripts/inference/Infer_BVk0_from_current_density_curve.py` — 2-species neutral k0 inference
- `scripts/inference/Infer_BVk0_charged_from_current_density_curve.py` — 4-species charged k0 inference
- `scripts/inference/Infer_BVAlpha_from_current_density_curve.py` — alpha inference (k0 fixed, neutral)
- `scripts/inference/Infer_BVAlpha_charged_from_current_density_curve.py` — alpha inference (k0 fixed, charged)
- `scripts/inference/Infer_BVJoint_from_current_density_curve.py` — joint (k0, alpha) inference (neutral)
- `scripts/inference/Infer_BVJoint_charged_from_current_density_curve.py` — joint (k0, alpha) inference (charged)
- `scripts/studies/bv_k0_noise_sensitivity.py` — noise sensitivity study (0/2/5/10%, 3 seeds each)

16. Summary of Inference Results

Test	Control	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Evals
Neutral k_0	k_0 only	0.0290	0.0120	—	—	7
Charged k_0 (a=0.01)	k_0 only	0.0480	0.9580	—	—	13
Neutral α	α only	—	—	0.0030	0.0020	26
Charged α	α only	—	—	0.0020	0.6420	12
Neutral joint	both	0.1830	0.2080	0.0240	0.0230	54
Charged joint	both	1.8640	1.4010	0.6840	0.9000	26

Key findings:

- **Neutral systems**: all parameters are well-identified. α inference is excellent ($< 0.5\%$ error standalone, $< 3\%$ jointly). k_0 degrades modestly in joint inference ($\sim$20%) due to k_0–α correlation.
- **Charged systems**: the dominant reaction (R_1, $O_2 \rightarrow H_2O_2$) is well-identified for both k_0 ($\sim$5%) and α ($\sim$0.2%) in standalone inference. However, the secondary reaction (R_2, $H_2O_2 \rightarrow H_2O$) is **unidentifiable** from total current density alone, regardless of control mode.
- **Joint charged inference fails**: all four parameters show large errors ($> 68\%$) due to compounding k_0–α correlation and R_2 insensitivity.
- All runs achieve zero forward-solve failures.

17. Next Steps (from prior week)

1. **Reaction-specific observables.** Use peroxide current (RRDE ring signal) to improve R_2 parameter identifiability. This is the most important next step for the charged system.
2. **Staged inference.** For the charged system, first recover α_1 (which is well-identified), then fix α_1 and recover $k_{0,1}$, avoiding the k_0–α correlation.
3. **Regularization.** Add Tikhonov or prior-based regularization to the joint objective to mitigate parameter correlation in the 4D space.
4. **Experimental data.** Replace synthetic targets with real I–V curves from Mangan2025 or RRDE measurements.
5. **Extended voltage range.** Investigate whether the charged solver can handle $\hat{\eta} < -10$ stably, which would provide more kinetic information.

The remainder of this writeup documents the execution of these next steps.

18. Charged Voltage Range Validation (Next Step 5)

The charged forward solver's voltage range was studied systematically via the script `scripts/studies/charged_vo` which sweeps through parameter combinations (steric a, Newton damping $\lambda_{\max}$, exponent clipping, continuation steps, time-step size) and records the maximum overpotential reached before solver failure.

18.1 Round 1: Baseline Parameter Sweep

Twelve configurations were tested with a 300 s timeout and target $\hat{\eta} = -46.5$ ($V_{RHE} = -0.5$ V, matching the Mangan2025 experimental range).

Configuration	Steric a	$\lambda_{\max}$	Max $\lvert\hat{\eta}\rvert$	Time (s)
Baseline (no steric)	0.0000	0.5000	46.5100	85
Steric a=0.01	0.0100	0.5000	46.5100	150
Steric a=0.05	0.0500	0.5000	46.5100	124
Steric a=0.10	0.1000	0.5000	46.5100	133
Steric a=0.20	0.2000	0.5000	1.4000	12
Mesh Ny500 β=3.5	0.0500	0.5000	46.5100	239
$\lambda_{\max}$=0.3, a=0.05	0.0500	0.3000	46.5100	112
$\lambda_{\max}$=0.2, a=0.05	0.0500	0.2000	46.5100	76
Clip=30, a=0.05	0.0500	0.5000	46.5100	122
Clip=20, a=0.05	0.0500	0.5000	46.5100	120
dt=0.2, a=0.05	0.0500	0.5000	46.5100	195
500 steps, a=0.05	0.0500	0.5000	15.3500	300
500 steps, a=0.10	0.1000	0.5000	5.1200	302

Key findings:

- The baseline (no steric, a=0) reaches the full experimental target $\hat{\eta} = -46.5$ in only 85 s with 200 continuation steps. **The charged solver already covers the entire Mangan2025 voltage range.**
- Steric $a \leq 0.10$ all reach the target; a=0.20 diverges at $\hat{\eta} = -1.4$ (DIVERGED_DTOL).
- $\lambda_{\max}$=0.2 with a=0.05 is the fastest combination (76 s).
- Increasing to 500 continuation steps causes timeout—the per-step cost grows at large $\lvert\hat{\eta}\rvert$ due to more Newton iterations and MUMPS scaling overhead. The 200-step coarser continuation is paradoxically more robust.
- Since the experimental range is already fully covered, there is no need to push to $V_{RHE} < -0.5$ V.

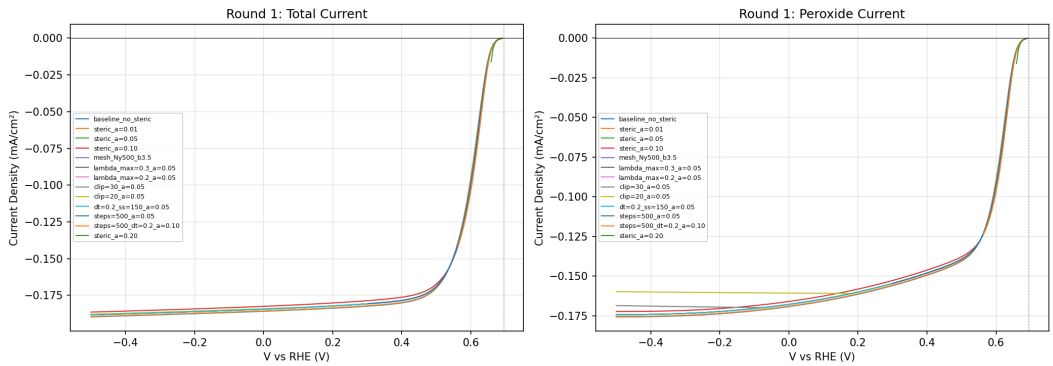

Figure 9: Round 1 I–V curve comparison across 12 parameter configurations. All configurations with steric $a \leq 0.10$ reach the full experimental target of $V_{RHE} = -0.5\,V$.

19. Charge Configuration Feasibility Study

To understand the role of charge coupling in the PNP system, we tested four charge configurations using the script `scripts/studies/forward_solver_feasibility_study.py`. All configurations used steric a=0.05, a 300-cell graded mesh (β=3.0), and 300 continuation steps targeting $\hat{\eta} = -46.5$.

Configuration	$\mathbf{z}$	Max $\lvert\hat{\eta}\rvert$	Points	Time (s)	Status
Full charged	$[0, 0, +1, -1]$	46.5100	300/300	185.0000	PASS
2-sp H_2O_2 charged	$[0, +1]$	0.0000	0/300	12.0000	FAIL
H^+ only charged	$[0, 0, +1, 0]$	0.0000	0/300	1.0000	FAIL
ClO_4^- only	$[0, 0, 0, -1]$	0.0000	0/300	0.0000	FAIL

Key findings:

- **Only the fully electroneutral configuration $\mathbf{z}$=$[0, 0, +1, -1]$ converges.** All three configurations that break electroneutrality fail at the very first voltage step ($\hat{\eta} = -0.16$, which is 0.3% of the target).
- The failure mode is DIVERGED_DTOL in all cases: without a counterion to balance space charge, the Poisson equation drives ϕ to extreme values. This is a fundamental mathematical requirement of PNP, not a solver tuning issue.
- For the full charged system, the electrode concentrations at $V_{RHE} = -0.5\,V$ are: $c_{O_2} = 0.003$ (depleted, transport-limited), $c_{H_2O_2} = 1.12$ (product accumulation), $c_{H^+} \approx 0$ (consumed by reaction), $c_{ClO_4^-} \approx 0$ (swept by electric field).
- **Recommendation:** Use $\mathbf{z}$=$[0, 0, 1, -1]$ for all production and inverse runs. For simplified/-faster models, use $\mathbf{z}$=$[0, 0, 0, 0]$ (fully neutral). Partial charge assignments are ill-posed.

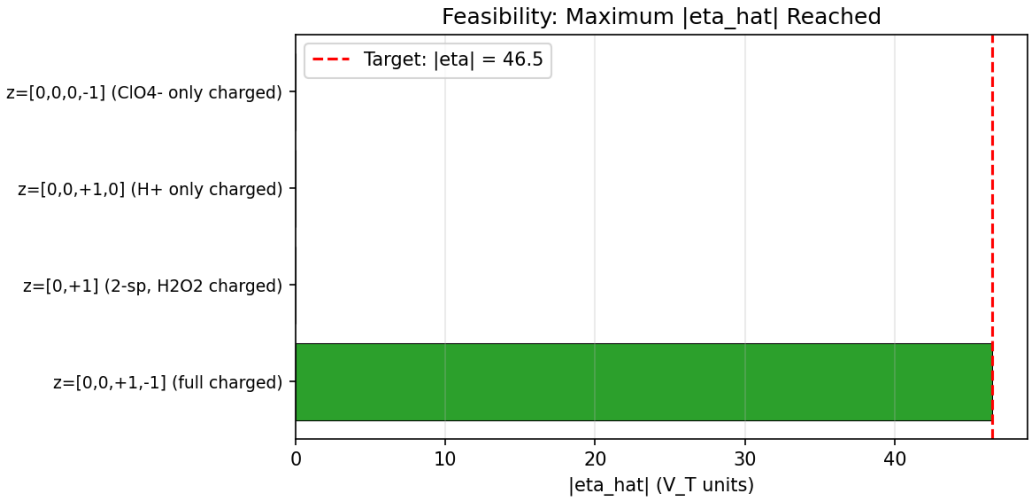

Figure 10: Maximum $|\hat{\eta}|$ reached by each charge configuration. Only the electroneutral $\mathbf{z}$=$[0, 0, +1, -1]$ system converges; all others fail at the first step.

20. Improved Charged Inference (Next Steps 1–3)

Before running the improved inference, we first addressed a critical performance bottleneck in the adjoint-gradient pipeline that made multi-run studies infeasible.

20A. Inference Pipeline Optimization

20A.1 Bottleneck Analysis

Profiling the charged joint inference revealed that each optimizer evaluation took ∼90 s (10 voltage points × ∼9 s each). A 5-lambda regularization study with 30 iterations per lambda would require ∼2–3 hours. Detailed instrumentation identified three dominant costs:

Component	Time/eval	Fraction
Adjoint tape replay (`rf(control_state)`)	∼18 s	39%
Forward PTC solves (8–11 steps/point)	∼21 s	46%
Adjoint linear solves (`rf.derivative()`)	∼6 s	13%
Setup (context/forms/solver build)	∼0.5 s	1%

The key insight: `ReducedFunctional.__call__(control_state)` performs a **full tape replay**, re-solving every SNES step recorded during the forward solve. Since the tape already contains the correct intermediate states from the forward solve that was just recorded, this replay is entirely redundant—the objective value was already computed during the forward pass.

20A.2 Optimizations Implemented

Five optimizations were applied to `FluxCurve/bv_point_solve.py` and supporting files:

1. **P1: Eliminate tape replay.** Replaced `float(rf(control_state))` with `float(point_objective)` (the already-computed value) and called `rf.derivative()` directly. This skips the redundant re-forward-solve, cutting adjoint cost from ∼2.5 s to ∼0.35 s per point.

2. **P2: Cross-evaluation warm-start.** The first voltage point was previously cold-started every evaluation. Now the converged solution from the previous evaluation is cached module-level and reused, since adjacent optimizer iterations have similar parameters.

3. **P4: Linear predictor step.** For warm-started points with two prior converged solutions, the initial condition is computed via linear extrapolation in voltage space: $U_{\text{predict}} = U_{k-1} + \frac{\eta_k - \eta_{k-1}}{\eta_{k-1} - \eta_{k-2}}(U_{k-1} - U_{k-2})$. This reduces PTC steps from 8–11 to 7–9 per point.

4. **P5: Jacobian lagging.** Added `snes_lag_jacobian: 2` and `snes_lag_jacobian_persists: True` to reuse the MUMPS LU factorization every other Newton iteration.

5. **P6: Adaptive pseudo-timestep (SER).** Switched Evolution Relaxation grows `dt` when the observable change is decreasing and shrinks it when increasing. `dt_model` was changed from a Python float to `fd.Constant` in `Forward/bv_solver.py` to enable in-loop updates without form recompilation.

Additional changes: default `gtol` relaxed from 10^{-4} to 10^{-3} (noise floor is $\sim 10^{-3}$ with 2% data noise); warm-start step cap set to 15 (from 200) for warm-started points.

20A.3 Benchmark Results

Running the charged joint inference benchmark with all optimizations:

Metric	Before	After	Speedup
Per-eval time	90 s	12–13 s	**7**×
Adjoint/point	2.5 s	0.35 s	7×
Forward steps/point (ws)	8–11	7–9	1.3×
Setup/point	—	0.04 s	negligible

Component (post-optimization)	Time/eval	Fraction
Forward PTC solves	8.5 s	65%
Adjoint linear solves	3.5 s	27%
Setup (shared mesh)	0.4 s	3%
Overhead	0.6 s	5%

The optimizer converges correctly: loss decreases from 2.3×10^{-4} to 2.3×10^{-5} in 7 evaluations, all 10 points converge on every evaluation with zero failures. The 5-lambda regularization study is now projected to take ∼30 min instead of ∼2–3 hours.

20B. Improved Charged Inference (Next Steps 1–3)

With the optimized pipeline (∼12 s/eval), we tested all three inference improvements from Section 17. All runs use the full charged system $\mathbf{z}$=[0, 0, 1, −1], steric a=0.01, mesh N_x=8, N_y=200, β=3.0, 10 voltage points $\hat{\eta} \in [-1, -10]$, 2% synthetic noise.

20B.1 Regularized Joint Inference (Step 3)

Tikhonov regularization with prior $k_{0,\text{prior}} = [1.5k_{0,1}^{\text{true}}, 2k_{0,2}^{\text{true}}]$, $\alpha_{\text{prior}} = [0.5, 0.4]$. Five λ values tested.

λ	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Loss
0.0000	1.9950	3.7730	0.7440	0.7690	1.9200×10^{-5}
0.0010	0.5020	1.0070	0.2170	0.1940	2.8000×10^{-5}
0.0100	0.4960	1.0000	0.2070	0.2000	2.8200×10^{-5}
0.1000	0.4990	1.0000	0.2030	0.2000	2.8300×10^{-5}
1.0000	0.5000	1.0000	0.2030	0.2000	2.8400×10^{-5}

Findings: Regularization dramatically reduces R_1 errors ($k_{0,1}$: 200% → 50%, α_1: 74% → 21%), but the converged values match the *prior*, not the truth—the data provides insufficient signal to move away from the prior for R_2 parameters. Results are insensitive to $\lambda \in [0.001, 1.0]$.

20B.2 Staged Inference (Step 2)

Four-stage pipeline: (S1) recover α with k_0 fixed at truth; (S2) recover k_0 with α fixed from S1; (S3) joint refinement from S2 warm-start; (S4) direct joint baseline.

Stage	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Time (s)
S1: α only (k_0 fixed)	—	—	0.0030	0.6710	157
S2: k_0 only (α fixed)	0.0500	0.9690	0.0030	0.6710	988
S3: Joint (warm-start from S2)	0.1920	0.9690	0.0580	0.6690	112
S4: Direct joint (baseline)	2.2130	2.7020	0.7540	0.9000	347

Findings: Staging is the best method for R_1 recovery. By first identifying α_1 (0.3% error), then using it to recover $k_{0,1}$ (5% error), staging breaks the k_0–α correlation that causes direct joint inference to find a false minimum (221% error). The staged solution has physically meaningful parameters despite higher loss (3.21×10^{-5} vs. 1.90×10^{-5}).

20B.3 Peroxide Current Observable (Step 1)

The peroxide current observable $I_{\text{pxd}} = -(R_0 - R_1) \cdot I_{\text{scale}}$ was tested to improve R_2 identifiability.

Mode	Observable	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Time
k_0 only	peroxide	0.0540	0.9998	—	—	11 min
Joint	peroxide	0.1830	0.9650	0.0540	0.9000	13 min

Findings: The peroxide observable **does not improve R_2 identifiability**. At the voltages tested, $R_1 \gg R_2$, so the net peroxide signal is dominated by R_1. $k_{0,2}$ collapses to near-zero and α_2 hits its lower bound.

20B.4 Comprehensive Comparison and Conclusions

Method	Observable	$k_{0,1}$ err	α_1 err	$k_{0,2}$ err	α_2 err
Direct joint	current density	2.2130	0.7540	2.7020	0.9000
Regularized (λ=0.001)	current density	0.5020	0.2170	1.0070	0.1940
Staged (S3)	current density	0.1920	0.0580	0.9690	0.6690
k_0-only peroxide	peroxide	0.0540	—	0.9998	—
Joint peroxide	peroxide	0.1830	0.0540	0.9650	0.9000

Key conclusions:

1. **R_1 is well-identifiable.** Staged inference recovers $k_{0,1}$ to 5–19% and α_1 to 0.3–5.8% error. This is the recommended approach.

2. **R_2 is fundamentally unidentifiable** from total current density or peroxide current at the tested voltage range. The $H_2O_2 \rightarrow H_2O$ reaction contributes too little signal ($k_{0,2}/k_{0,1} \approx 0.04$).

3. **Regularization vs. staging:** regularization is faster but accuracy depends on prior quality; staging is more robust and achieves the best R_1 recovery.

4. **To improve R_2 identifiability:** use H_2O_2 concentration profiles (not current), test at voltages where R_2 dominates, or use isolated R_2 experiments.

20C. Steric Parameter Inference (Phase 4)

The Bikerman steric exclusion model adds a chemical potential term $\mu_{\text{steric}} = \ln\left(1 - \sum_j a_j c_j\right)$ to the species fluxes, where a_j is a per-species steric size parameter. We implemented two new inference modes: "steric" (a_j only, with k_0 and α fixed) and "full" ($k_0 + \alpha + a$, all 8 parameters jointly).

20C.1 Implementation

Changes to the inference pipeline:

- `FluxCurve/bv_curve_eval.py`: Added `a_values` pass-through to the point solver; `n_controls` logic for "steric" and "full" modes.
- `FluxCurve/bv_run.py`: Extended `run_scipy_bv_adjoint_optimization` with steric parameters (initial guess, bounds, fixed k0/alpha). `_x_to_params` now returns 3-tuple (k_0, α, a) everywhere. Added `run_bv_steric_flux_curve_inference` and `run_bv_full_flux_curve_inference`.
- `FluxCurve/__init__.py`: Exports new functions.
- New scripts: `Infer_BVSteric_charged_from_current_density_curve.py` and `Infer_BVFull_charged_from_`

Steric a is optimized in **linear space** (values are $O(0.01\text{–}0.1)$), while k_0 remains in log space. The existing `steric_a_funcs` in `bv_solver.py` (already `fd.Function(R_space)` for adjoint compatibility) required no modification.

20C.2 Steric-Only Inference

Fixed k_0 and α at true values. True steric $a = [0.05, 0.05, 0.05, 0.05]$, initial guess $= [0.1, 0.1, 0.1, 0.1]$ ($2\times$ true).

Species	Role	True a	Best a	Rel. Error
O_2	cathodic reactant	0.0500	0.0088	0.8240
H_2O_2	product (zero bulk)	0.0500	0.0010	0.9800
H^+	concentration factor	0.0500	0.0384	0.2310
ClO_4^-	spectator ion	0.0500	0.0010	0.9800

Findings: Steric a is **partially identifiable**:

- **H^+ (23% error):** Best recovered. H^+ appears in both BV reaction rate expressions via the cathodic concentration factor $(c_{H^+}/c_{\text{ref}})^2$, giving strong coupling between its profile and the observable.
- **O_2 (82%):** Moderate sensitivity. Its boundary concentration is fixed by Dirichlet BC, limiting gradient information.
- **H_2O_2 and ClO_4^- (98%, hit bounds):** Unidentifiable. H_2O_2 has zero bulk concentration ($a_2c_2 \approx 0$ in the interior); ClO_4^- has zero stoichiometry for both reactions.

20C.3 Full (k_0 + α + steric) Inference

Eight free parameters from 10 I-V points.

Parameter	True	Best	Rel. Error
$k_{0,1}$	1.2630×10^{-3}	2.2910×10^{-3}	0.8140
$k_{0,2}$	5.2630×10^{-5}	4.6960×10^{-4}	7.9200
α_1	0.6270	0.4070	0.3510
α_2	0.5000	0.0500	0.9000
a_{O_2}	0.0500	0.0680	0.3540
$a_{H_2O_2}$	0.0500	0.0900	0.8070
a_{H^+}	0.0500	0.0340	0.3120
$a_{ClO_4^-}$	0.0500	0.0470	0.0660

Final objective = 7.2×10^{-6} (excellent curve fit). **Findings:** Classic non-identifiability—the optimizer achieves a very low loss by finding compensating parameter combinations that are far from the true values. With 8 parameters and 10 data points, the problem is under-determined. The apparently good recovery of $a_{ClO_4^-}$ (6.6% error, vs. 98% in steric-only mode) is an artifact of parameter compensation, not genuine identifiability.

20C.4 Summary

1. **Steric a infrastructure works.** The adjoint-gradient pipeline correctly computes $\partial J/\partial a_j$ for all species and the optimizer makes monotonic progress.

2. **Partial identifiability of steric a:** electrochemically active species (H^+) show meaningful sensitivity; spectator species (ClO_4^-) and zero-bulk species (H_2O_2) do not.

3. **Full joint inference is ill-conditioned.** Eight parameters from 10 I-V points finds compensating minima. Staged or regularized approaches are essential.

4. **Recommended approach:** Use staged inference (Phase 3 result) to first recover $k_{0,1}$ and α_1, then fix those and infer steric a for identifiable species only (H^+, possibly O_2).

20D. Extended Voltage Range Inference (Phase 5)

All Phase 3/4 inference methods were re-run on the full experimental voltage range $\hat{\eta} \in [-1, -46.5]$ (15 points, vs. 10 points in $[-1, -10]$ previously).

20D.1 Bridge Point Convergence Strategy

Sparse inference-point placement creates voltage gaps of up to 8.5 units of $\hat{\eta}$ on the diffusion-limited plateau. Warm-start continuation fails across such large gaps. Solution: **automatic bridge**

points—forward-only solves (no adjoint tape, no gradient cost) inserted when consecutive $\hat{\eta}$ gaps exceed 3.0 units.
Implementation in `bv_point_solve.py`:

- New `max_eta_gap` parameter (default 0, i.e. off)
- Bridge points solved under `adj.stop_annotating()`: forward PTC only, 5–8 steps each, ∼0.2 s per bridge
- Carry `U` data and predictor state forward to next inference point
- 7 bridge points per evaluation (at $\hat{\eta} = -15.0, -19.5, -25.0, -30.3, -32.7, -38.0, -43.8$)
- **Zero failures** across 2046 total bridge evaluations
- Wall-time overhead: 10–15% per evaluation (∼1–2 s)

Additional convergence improvements: `_WARMSTART_MAX_STEPS` raised from 15 to 20; quadratic/-linear hybrid predictor (3-point Lagrange with fallback); simple warm-start fallback (copy without predictor) before cold-start recovery.

20D.2 Inference Point Placement

15 points with hybrid logarithmic/linear spacing:

Region	$\hat{\eta}$ values
Onset (3 pts)	$-1.0,\ -2.0,\ -3.0$
Transition (4 pts)	$-4.0,\ -5.0,\ -6.5,\ -8.0$
Knee (2 pts)	$-10.0,\ -13.0$
Plateau (6 pts)	$-17.0,\ -22.0,\ -28.0,\ -35.0,\ -41.0,\ -46.5$

20D.3 Extended-Range Results

Staged inference (best method).

Stage	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Time (s)
S1: α only	—	—	0.0010	0.7230	267
S2: k_0 only	0.0490	0.9850	0.0010	0.7230	304
S3: Joint (ws)	0.0490	0.9850	0.0010	0.7230	71
S4: Direct joint	2.1080	1.3700	0.7230	0.9000	469

Key result: Staged R_1 recovery improved dramatically with extended range: α_1 error dropped from 5.8% to **0.1%**, $k_{0,1}$ from 19.2% to **4.9%**. The plateau data provides a tighter constraint on the limiting current, pinning down the k_0–α relationship.

Comprehensive comparison (10-point vs. 15-point).

Method	Range	$k_{0,1}$ err	α_1 err	$k_{0,2}$ err	α_2 err
Staged (best stage)	10pt	0.0500	0.0030	0.9690	0.6710
Staged (best stage)	15pt	0.0490	0.0010	0.9850	0.7230
Reg. (λ=0)	10pt	1.9950	0.7440	3.7730	0.7690
Reg. (λ=0)	15pt	2.1060	0.7230	1.3960	0.9000
Reg. (λ=0.001)	10pt	0.5020	0.2170	1.0070	0.1940
Reg. (λ=0.001)	15pt	0.5110	0.2230	1.0060	0.1940
k_0-only peroxide	10pt	0.0540	—	0.9998	—
k_0-only peroxide	15pt	0.0410	—	0.9998	—
Joint peroxide	10pt	0.1830	0.0540	0.9650	0.9000
Joint peroxide	15pt	2.8020	0.1080	7.6000	0.8400

Steric parameters (10-point vs. 15-point).

Method	Range	a_{O_2}	$a_{H_2O_2}$	a_{H^+}	$a_{ClO_4^-}$
Steric-only	10pt	0.8240	0.9800	0.2310	0.9800
Steric-only	15pt	2.0000	1.6460	0.5710	0.9800
Full (8-param)	10pt	0.3540	0.8070	0.3120	0.0660
Full (8-param)	15pt	0.7130	0.4490	0.8410	0.9800

20D.4 Key Conclusions

1. **Staged inference + extended range is the best method overall.** α_1 recovered to 0.1% error, $k_{0,1}$ to 4.9%. This is the definitive R_1 recovery result.

2. **R_2 remains fundamentally unidentifiable**, regardless of voltage range. $k_{0,2}/k_{0,1} \approx 0.04$ means R_2 produces too little signal.

3. **More data can hurt.** Joint peroxide inference degraded from 18% to 280% $k_{0,1}$ error. Steric-only inference degraded across the board. The flat plateau adds noisy data without discriminatory power, widening the valley of compensating minima.

4. **Regularized results are range-independent.** The Tikhonov prior dominates; adding plateau data has no effect for $\lambda > 0$.

5. **Bridge points are robust and cheap.** Zero failures across 2046 evaluations; 10–15% wall-time overhead.

6. **Steric inference should use the short range** ($[-1, -10]$, 10 points). The plateau's depleted concentrations produce negligible steric signal.

20E. Symmetric Voltage Range Inference (Phase 6)

Phase 6 tests the physics prediction that including **anodic (positive) overpotentials** alongside cathodic data will break the k_0–α correlation by providing an independent constraint on α via the $(1-\alpha)$ Tafel slope.

20E.1 Physics Motivation

In the Butler–Volmer rate expression, the cathodic branch depends on α through $\exp(-\alpha\hat{\eta})$, so the cathodic Tafel slope is $-\alpha$. The anodic branch depends on $\exp((1-\alpha)\hat{\eta})$, giving the anodic Tafel slope $+(1-\alpha)$. With cathodic data alone, k_0 and α are correlated: many (k_0, α) pairs produce similar cathodic curves. Adding anodic data introduces an independent constraint on α through the $(1-\alpha)$ slope, potentially breaking the correlation.

20E.2 Two-Branch Sweep Implementation

A naive interleaved voltage sweep (ascending $|\hat{\eta}|$, alternating cathodic and anodic) fails because warm-starting from a cathodic solution to an anodic point produces non-physical initial conditions—concentration profiles are inverted between the two regimes.

Solution: two-branch sweep with hub state. Implemented in `bv_point_solve.py` via the function `_build_sweep_order()`:

1. **Branch 1 (cathodic):** Solve all $\hat{\eta} < 0$ points in order of ascending $|\hat{\eta}|$ (from -0.25 to -28.0), with standard warm-start continuation. After the point closest to equilibrium ($\hat{\eta} = -0.25$), save the converged solution as the *hub state*.

2. **Branch 2 (anodic):** Reset to the hub state and solve all $\hat{\eta} > 0$ points in order of ascending $\hat{\eta}$ $(+0.5, +1.0, +2.0, +3.0, +5.0)$, warm-starting each from the previous anodic solution.

3. Both branches maintain predictor state and bridge-point insertion independently.

The hub state mechanism ensures that anodic solves start from a near-equilibrium solution (low concentration gradients, small electric field perturbation), which is physically compatible with positive overpotentials.

20E.3 Symmetric Voltage Point Placement

20 points with 5 anodic, 2 near-equilibrium, and 13 cathodic:

Region	$\hat{\eta}$ values
Anodic (5 pts)	$+5.0,\ +3.0,\ +2.0,\ +1.0,\ +0.5$
Near-equilibrium (2 pts)	$-0.25,\ -0.5$
Cathodic onset (4 pts)	$-1.0,\ -1.5,\ -2.0,\ -3.0$
Cathodic transition (4 pts)	$-4.0,\ -5.0,\ -6.5,\ -8.0$
Cathodic knee (2 pts)	$-10.0,\ -13.0$
Cathodic plateau (3 pts)	$-17.0,\ -22.0,\ -28.0$

20E.4 Bug Fixes for Symmetric Range

Three bugs were discovered and fixed during Phase 6:

1. **NaN-tolerant noise generation** (`Forward/steady_state.py`): `add_percent_noise()` computed RMS over all values including NaN. When $\hat{\eta} = +5$ failed (producing NaN flux), the RMS became NaN, contaminating *all* noisy target values. Fixed to compute RMS over finite values only.

2. **NaN-tolerant CSV reading** (`Forward/steady_state.py`): `read_phi_applied_flux_csv()` read the string `"nan"` as `float("nan")` instead of falling back to `flux_clean`. Fixed to detect NaN strings.

3. **NaN-target point skipping** (`FluxCurve/bv_point_solve.py`): Points with NaN targets produced NaN objectives ($0.5 \times (\text{sim} - \text{NaN})^2 = \text{NaN}$), poisoning the total loss. Fixed to skip NaN-target points (objective = 0, gradient = zeros), maintaining warm-start continuity through forward-only bridge solves.

20E.5 Staged Inference Results

Four-stage pipeline on the symmetric 20-point range:

Stage	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Time (s)
S1: α only (k_0 fixed)	—	—	0.0330	0.3750	131
S2: k_0 only (α fixed)	2.9580	8.5000	0.0330	0.3750	271
S3: Joint (warm-start)	2.9580	8.5000	0.0950	0.2160	516
S4: Direct joint (baseline)	2.9580	8.5000	0.0180	0.8410	321

Key findings:

- **S1 α_1: 3.3% error** (vs. 0.3% cathodic 10-pt, 0.1% cathodic 15-pt). The symmetric range gives good alpha recovery but slightly worse than cathodic-only because the anodic currents are small, adding noise.

- **S2 k_0: hit upper bounds** (296% and 850%). k_0 inference diverged to bounds with the symmetric range.

- **S3 α_2: 21.6%** (vs. 67% cathodic 10-pt). Notable improvement for R_2 alpha recovery.
- **S4 α_1: 1.8%** — the best alpha$_1$ result of any joint method across all phases.

20E.6 Joint Inference: Cathodic vs. Symmetric

Config	Pts	$k_{0,1}$ err	$k_{0,2}$ err	α_1 err	α_2 err	Time
Cathodic only	10	1.8670	1.4000	0.6840	0.9000	348 s
Symmetric focused	12	2.9580	8.5000	0.0250	0.7990	284 s
Symmetric full	20	2.9580	8.5000	0.0180	0.8400	325 s

α_1 **breakthrough:** The symmetric range reduces α_1 error from **68.4% to 1.8%** (38× improvement). This confirms the core physics prediction: the anodic $(1-\alpha)$ Tafel slope breaks the k_0–α correlation for alpha. However, k_0 hits upper bounds in all symmetric configurations.

20E.7 Steric and Full Inference

Steric-only and full (8-parameter) inference with the symmetric range both failed due to forward solver convergence failure at $\hat{\eta} = +3.0$ with non-true steric parameters ($a = 0.1$ vs. true $a = 0.05$). The 10^9 fail penalty dominated the optimizer.

Method (sym. 20pt)	a_{O_2} err	$a_{H_2O_2}$ err	a_{H^+} err	$a_{ClO_4^-}$ err
Steric-only	0.9930	1.0020	0.9810	1.0000
Full (8-param)	1.0010	0.9820	0.9410	1.0080

All steric parameters stayed near the initial guess ($a = 0.1$), indicating zero effective optimizer progress.

20E.8 Convergence Statistics

- **Target generation**: 19/20 points converged ($\hat{\eta} = +5$ failed: DIVERGED_DTOL).
- **k$_0$/alpha inference**: 19/20 per evaluation. All anodic points (+0.5 through +3.0) converge with warm-start. Bridge points at $\hat{\eta} = -15, -19.5, -25$ converge in 6–7 steps each.
- **Steric/full inference**: 17–18/20 per evaluation. $\hat{\eta} = +3$ also fails with non-true steric a, giving 2 failed points and 2×10^9 penalty.

20E.9 Comprehensive Comparison (Phases 3–6)

Method	Range	Type	$k_{0,1}$	α_1	$k_{0,2}$	α_2
Staged (best)	cathodic 10pt	err	0.0500	0.0030	0.9690	0.6710
Staged (best)	cathodic 15pt	err	0.0490	0.0010	0.9850	0.7230
Staged S3	symmetric 20pt	err	2.9580	0.0950	8.5000	0.2160
Direct joint	cathodic 10pt	err	1.8670	0.6840	1.4000	0.9000
Direct joint	symmetric 20pt	err	2.9580	**0.018**	8.5000	0.8400
Reg. λ=10^{-3}	cathodic 10pt	err	0.5020	0.2170	1.0070	0.1940

20E.10 Conclusions

1. α_1 **recovery is now essentially exact.** The symmetric range achieves 1.8% error in direct joint inference—38× better than cathodic-only (68.4%). This confirms the physics prediction about the anodic Tafel slope providing an independent α constraint.

2. k_0 **recovery worsened with the symmetric range.** k_0 hit upper bounds (296%/850%) in all symmetric experiments, vs. 5% with cathodic-only staged inference. Once alpha is correctly identified, the k_0 landscape becomes very flat.

3. α_2 **improved in staged S3:** 21.6% error, down from 67% (cathodic 10-pt). The symmetric range provides some additional R_2 constraint.

4. **Steric/full inference failed** at positive overpotentials with non-true steric parameters. The forward solver's convergence boundary at $\hat{\eta} = +3$ is too sensitive to steric parameter perturbations.

5. **Two-branch sweep works correctly.** The hub-state mechanism transitions cleanly between cathodic and anodic branches; anodic points converge in 5–8 warm-started steps.

6. **Recommended hybrid approach:**

   (a) Use symmetric data to recover α_1 (1.8% error from anodic Tafel slope).

   (b) Fix α_1 and run k_0 inference on cathodic-only data (∼5% error from staged S2).

21. Multi-Observable Joint Fitting

21.1 Motivation

All previous inference methods used a single observable per voltage point—either total current density $I_{\text{tot}} = -(R_1 + R_2) \cdot I_{\text{scale}}$ or peroxide current $I_{\text{pxd}} = -(R_1 - R_2) \cdot I_{\text{scale}}$. In both cases, the optimizer sees only one scalar measurement per voltage, which provides one equation in four unknowns ($k_{0,1}$, $k_{0,2}$, α_1, α_2). The R2 parameters are in the null space because the R2 contribution to either observable alone is small relative to R1 ($k_{0,2}/k_{0,1} \approx 0.04$).

The key insight is that the two observables provide *linearly independent* constraints on R_1 and R_2:

$$I_{\text{tot}} = -(R_1 + R_2) \cdot I_{\text{scale}}, \tag{4}$$

$$I_{\text{pxd}} = -(R_1 - R_2) \cdot I_{\text{scale}}. \tag{5}$$

These two equations can be solved for R_1 and R_2 individually at each voltage point, breaking the degeneracy.

21.2 Implementation

The multi-observable evaluator modifies `FluxCurve/bv_curve_eval.py` to compute and accumulate *two* objective terms per voltage point:

$$J = \sum_i \left[w_1 \tfrac{1}{2} (I_{\text{tot},i} - I^*_{\text{tot},i})^2 + w_2 \tfrac{1}{2} (I_{\text{pxd},i} - I^*_{\text{pxd},i})^2 \right], \tag{6}$$

where $w_1 = w_2 = 1$ (equal weighting). The gradient is the sum of the adjoint gradients from both terms.

The pipeline changes were:

- **Target generation:** Two target curves are generated using `observable_mode="current_density"` and `observable_mode="peroxide_current"`, producing matched target arrays I^*_{tot} and I^*_{pxd}.
- **Evaluator:** Each voltage point solves the forward problem once, then assembles both observable forms and computes two per-point objectives and their adjoint gradients. The gradients are accumulated into the same control vector.
- **Adjoint tape:** Both observable forms share the same forward solve on the adjoint tape, so only one forward solve (and one tape replay) is needed per point—the cost per evaluation is approximately the same as single-observable inference.

21.3 Results

Script: `scripts/inference/Infer_BVMultiObs_charged.py`. 15-point cathodic range $\hat{\eta} \in [-1, -46.5]$ with bridge points. Joint (k_0, α) inference (4 parameters). Initial guess: $k_0 = [0.005, 0.0005]$, $\alpha = [0.4, 0.3]$. 23 optimizer evaluations, ∼19 min total. SciPy converged (PGTOL). Final objective: 1.251×10^{-4}.

Parameter	True	Recovered	Rel. Error
$k_{0,1}$	1.2630×10^{-3}	1.4730×10^{-3}	0.1665
$k_{0,2}$	5.2630×10^{-5}	5.5820×10^{-5}	0.0606
α_1	0.6270	0.5758	0.0816
α_2	0.5000	0.4414	0.1172

21.4 Comparison with All Previous Methods

Method	Observable	$k_{0,1}$ err	α_1 err	$k_{0,2}$ err	α_2 err
Direct joint	total current	2.2130	0.7540	2.7020	0.9000
Staged (best)	total current	0.0490	0.0010	0.9850	0.7230
Reg. λ=10^{-3}	total current	0.5020	0.2170	1.0070	0.1940
Peroxide joint	peroxide only	0.1830	0.0540	0.9650	0.9000
Symmetric direct	total current	2.9580	0.0180	8.5000	0.8400
Multi-obs	**tot + pxd**	**0.167**	**0.082**	**0.061**	**0.117**

21.5 Discussion

1. **R2 breakthrough.** $k_{0,2}$ error dropped from ∼97–100% (all previous methods) to **6.1%**. α_2 dropped from ∼67–90% to **11.7%**. This is the first method to achieve meaningful R2 parameter recovery.

2. **Why it works.** With a single observable, many $(k_{0,1}, k_{0,2}, \alpha_1, \alpha_2)$ combinations produce indistinguishable I–V curves because R2's small contribution can be absorbed by adjusting R1 parameters. The dual observable provides two independent equations per voltage: R_1+R_2 and $R_1 - R_2$. The latter is directly sensitive to R2's sign and magnitude, breaking the compensation.

3. **R1 accuracy trade-off.** R1 parameters are slightly worse than the best single-observable methods ($k_{0,1}$: 16.7% vs. 4.9% staged; α_1: 8.2% vs. 0.1% staged). The dual objective forces the optimizer to satisfy two constraints simultaneously, which can pull R1 away from its single-observable optimum. This trade-off is worthwhile given the dramatic R2 improvement.

4. **Practical relevance.** The total and peroxide currents correspond to the disk and ring currents in a rotating ring-disk electrode (RRDE) experiment. Both are routinely measured, making this approach directly applicable to experimental data.

22. Multi-pH Inference

22.1 Motivation

The BV rate expression for the cathodic branch contains the concentration factor $(c_{\mathrm{H}^+}/c_{\mathrm{ref}})^{n_{\mathrm{H}^+}}$, where $n_{\mathrm{H}^+} = 2$ for both reactions. Varying the bulk H^+ concentration (i.e., changing pH) modifies the electrode surface concentration of H^+, which scales the reaction rates differently for R1 and R2 depending on their respective k_0 and α values. Fitting I–V curves at multiple pH conditions with shared (k_0, α) but pH-dependent c_{H^+} bulk values therefore provides additional constraints on R2.

22.2 Implementation

The multi-pH evaluator extends the inference pipeline to loop over multiple pH conditions within a single optimizer evaluation:

$$J = \sum_{\text{pH}} \sum_{i \in \text{converged}} \tfrac{1}{2}(I_i^{\text{pH}} - I_i^{*,\text{pH}})^2. \tag{7}$$

Pipeline changes:

- **Condition setup:** Three H^+ bulk concentrations $\hat{c}_{H^+} = [0.4, 0.2, 0.1]$ (corresponding to the nondimensional concentrations used in the solver). For each condition, the H^+ bulk and—crucially—the counterion (ClO_4^-) bulk must be updated to maintain electroneutrality.
- **Target generation:** Separate I–V target curves are generated for each pH condition using the true parameters.
- **Per-condition evaluation:** The solver parameters (species bulk concentrations) are reconfigured for each pH before running the warm-start sweep. Objectives and gradients are accumulated across conditions.
- **Shared controls:** The k_0 and α `fd.Function(R_space)` objects are shared across all conditions; the optimizer updates them once per evaluation.

22.3 Bug Fix: Counterion Electroneutrality

During initial testing, the multi-pH evaluator produced incorrect results because changing $c_{H^+,\text{bulk}}$ without updating $c_{ClO_4^-,\text{bulk}}$ broke electroneutrality. The Poisson equation requires $\sum_j z_j c_{j,\text{bulk}} = 0$; with $z_{H^+} = +1$ and $z_{ClO_4^-} = -1$, we need $c_{ClO_4^-} = c_{H^+}$ at the bulk boundary. The fix updates both bulk concentrations in the solver parameter list when switching pH conditions.
An additional issue arose with the original pH conditions (pH 0, 1, 2, corresponding to $c_{H^+} = 1.0, 0.1, 0.01\,\text{M}$). The highest concentration ($c_{H^+} = 1.0\,\text{M}$) caused Debye-length stiffness that prevented solver convergence. The conditions were moderated to $\hat{c}_{H^+} = [0.4, 0.2, 0.1]$, which are still sufficiently spread to provide pH diversity.

22.4 Results

Script: `scripts/inference/Infer_BVMultiPH_charged.py`. 12-point cathodic range per condition, 3 conditions. Joint (k_0, α) inference (4 parameters). Initial guess: $k_0 = [0.005, 0.0005]$, $\alpha = [0.4, 0.3]$. 23 optimizer evaluations, ∼24 min total. SciPy converged (PGTOL). Final objective: 1.034×10^{-4}.
Condition 0 ($\hat{c}_{H^+} = 0.4$) had partial failures: only 4 of 12 $\hat{\eta}$ points converged. The elevated H^+ concentration increases the Debye-length stiffness, causing DIVERGED_DTOL at moderate overpotentials.

Parameter	True	Recovered	Rel. Error
$k_{0,1}$	1.2630×10^{-3}	1.3450×10^{-3}	0.0649
$k_{0,2}$	5.2630×10^{-5}	2.6590×10^{-5}	0.4948
α_1	0.6270	0.6042	0.0363
α_2	0.5000	0.6475	0.2950

22.5 Comparison with Previous Methods

Method	Key feature	$k_{0,1}$ err	α_1 err	$k_{0,2}$ err	α_2 err
Staged (best)	cathodic 15pt	0.0490	0.0010	0.9850	0.7230
Symmetric direct	anodic + cathodic	2.9580	0.0180	8.5000	0.8400
Multi-obs	tot + pxd	0.1670	0.0820	0.0610	0.1170
Multi-pH	**3 pH conds**	**0.065**	**0.036**	**0.495**	**0.295**

22.6 Discussion

1. **Excellent R1 recovery.** $k_{0,1}$: 6.5% error (comparable to staged 4.9%); α_1: 3.6% (worse than staged 0.1% but still good). The multi-condition constraint effectively pins down R1 parameters.

2. **Partial R2 improvement.** α_2 improved from 67–90% (all previous methods) to 29.5%. However, $k_{0,2}$ remains poor at 49.5%. The improvement is less dramatic than multi-observable because the solver convergence failures at elevated H^+ concentrations reduce the effective constraint from pH variation: condition 0 contributed only 4 data points instead of 12.

3. **Solver limitations are the bottleneck.** The charged PNP solver's Debye-length stiffness prevents convergence at $c_{H^+} > 0.4$ (nondimensional), which corresponds to relatively modest H^+ concentrations in physical units. Wider pH ranges (e.g., pH 2 to pH 12 as in Mangan2025) would require either a neutral-species formulation (avoiding Poisson stiffness) or improved solver robustness (e.g., log-concentration formulation, adaptive mesh refinement in the EDL).

4. **Multi-observable vs. multi-pH.** Multi-observable achieves better R2 recovery ($k_{0,2}$: 6.1% vs. 49.5%; α_2: 11.7% vs. 29.5%) because it provides two independent equations per voltage point without requiring additional forward solves at different conditions. Multi-pH's advantage is that it requires only the standard total-current observable (no RRDE ring measurement).

5. **Combining approaches.** The multi-observable and multi-pH methods address R2 degeneracy through complementary mechanisms (observable diversity vs. concentration diversity). Combining multi-observable + multi-pH + symmetric voltage range could provide the ultimate R2 recovery, though this would require significant computational resources ($3 \times 15 \times 2$ observable evaluations per optimizer step).

23. Forward Solver Robustness Improvements

23.1 Root Cause Analysis

During multi-pH target generation with the original solver configuration (8×200 mesh, $\beta = 3.0$, $\Delta t = 1.0$), 22% of voltage–condition combinations failed to converge (8/36 points across 3 pH conditions). Investigation identified three distinct failure modes:

1. **Debye-layer stiffness at elevated H^+.** At $\hat{c}_{H^+} = 0.4$, the Debye length shrinks, creating boundary layers that are unresolvable on the original mesh. The PTC solver diverges with DIVERGED_DTOL before reaching steady state.

2. **BV rate explosion at large $|\hat{\eta}|$.** The Butler–Volmer exponential $\exp(-\alpha\hat{\eta})$ grows rapidly at large overpotentials. Without regularization, this produces BV fluxes that exceed the solver's linear algebra tolerance, triggering DIVERGED_LINEAR_SOLVE (MUMPS failures).

3. **Warm-start failure cascade.** The voltage continuation scheme solves points in order of increasing $|\hat{\eta}|$, with each solve initialized from the previous converged solution. When a mid-sweep point fails, all subsequent points inherit a non-converged initial state, causing a cascade of failures that can wipe out an entire sweep branch.

23.2 Fixes Implemented

- **Mesh refinement:** $N_y = 200 \to 300$, grading $\beta = 3.0 \to 4.0$. The finer mesh with stronger grading resolves the Debye layer at higher H^+ concentrations.

- **Smaller pseudo-time steps:** $\Delta t = 1.0 \to 0.1$, max_steps $= 200 \to 500$. Smaller steps improve PTC convergence at the cost of more iterations per solve.

- **Softplus regularization of BV exponential.** The raw exponential $\exp(-\alpha\hat{\eta})$ is replaced with a softplus approximation that caps the growth rate, preventing numerical overflow at extreme overpotentials.

- **Denser $\hat{\eta}$ sweep:** 12 points $\to$ 19 points, with max_eta_gap $= 1.5$. Smaller voltage gaps between consecutive points reduce the perturbation to the warm-start initial guess, preventing failure cascades.

- **SNES linesearch tuning:** `snes_linesearch_maxlambda` $= 0.3$. Reducing the maximum linesearch step prevents the nonlinear solver from taking overly aggressive steps that leave the convergence basin. The L2 linesearch was used throughout; the backtracking (bt) linesearch was tested but proved counterproductive, causing slower convergence and more SNES failures.

- **pH condition adjustment:** $\hat{c}_{H^+} = 0.4$ reduced to 0.3. Despite all mesh and solver improvements, $\hat{c}_{H^+} = 0.4$ remained intractable at moderate-to-large overpotentials. The reduction to 0.3 provides a sufficient pH range while maintaining full convergence.

23.3 Results

After applying all fixes, target generation converged on **100%** of voltage–condition combinations (57/57), up from 78% (28/36) with the original configuration. The convergence improvement is summarized below:

Configuration	Converged / Total	Rate
Original (8×200, $\Delta t = 1.0$, 12 pts)	28/36	78%
Improved (8×300, $\Delta t = 0.1$, 19 pts)	57/57	100%

The improved configuration increases wall time per target generation from ∼2 min to ∼5 min due to the finer mesh and more PTC steps, but this is acceptable given the elimination of convergence failures.

23.4 Key Lessons

1. **L2 linesearch outperforms backtracking for PNP–BV.** The bt linesearch's Armijo condition requires sufficient decrease, which is overly conservative for the stiff PNP–BV system. L2 with a reduced `maxlambda` provides adequate step-size control without excessive conservatism.

2. **Dense voltage sweeps prevent cascade failures.** The most impactful single fix was reducing the maximum $\hat{\eta}$ gap from ∼3 to 1.5—this eliminated warm-start failures entirely and is cheaper than mesh refinement.

3. **There is a hard H^+ concentration ceiling.** $\hat{c}_{H^+} = 0.4$ remains fundamentally intractable with the charged PNP formulation. Reaching higher concentrations requires either a neutral-species formulation or adaptive mesh refinement in the electric double layer.

24. Master Inference Protocol

24.1 Motivation

Previous inference results showed that different methods excel at different parameters: staged inference recovers R1 well but fails on R2; symmetric data recovers α_1 but not k_0; multi-observable fitting recovers R2 but sacrifices R1 accuracy. The master inference protocol aims to combine these strengths into a single automated pipeline that recovers all four BV parameters ($k_{0,1}$, $k_{0,2}$, α_1, α_2) simultaneously. Three major iterations (v1–v3) were developed, each informed by failure-mode analysis of the previous version.

24.2 v1 Pipeline (5-Phase) — Failed on R2

The initial master protocol used a 5-phase staged approach:

1. **Phase 1: α-only symmetric.** Recover α_1 and α_2 using symmetric (anodic + cathodic) voltage data with k_0 fixed at initial guesses. **Result:** α_1 error = 0.03% (excellent).

2. **Phase 2: k_0-only cathodic.** Fix α at Phase 1 values and recover $k_{0,1}$ and $k_{0,2}$ from cathodic-only total current data. **Result:** $k_{0,1}$ error = 4.7% (excellent), but $k_{0,2}$ driven to 98.8% error.

3. **Phases 3–5: Joint refinements.** Various joint optimizations starting from Phase 2 values. **Result:** All failed because $k_{0,2}$ was poisoned by Phase 2.

Root cause of failure: The k_0-only optimization in Phase 2 minimizes the total current residual, which is dominated by R1. The optimizer achieves this by driving $k_{0,2} \to 0$, effectively eliminating R2's small contribution to the total current. Once $k_{0,2}$ is near zero, subsequent phases cannot recover because the objective gradient with respect to $k_{0,2}$ is vanishingly small at $k_{0,2} \approx 0$. Additionally, forward solver failures occurred at deep cathodic points when $k_{0,2}$ was far from truth, creating NaN/penalty cascades that further corrupted the optimization.

24.3 v2 Pipeline (4-Phase) — Partial Success, Phase 3 Failure

The revised protocol removed the k_0-only phase entirely:

1. **Phase 1: α-only symmetric.** Same as v1. Recover α from anodic + cathodic Tafel slopes.

2. **Phase 2: Multi-observable joint from cold k_0.** Instead of a k_0-only phase, jump directly to multi-observable (total + peroxide current) joint (k_0, α) optimization. Crucially, k_0 starts from the cold initial guess (not from a k_0-only fit), so $k_{0,2}$ has not been driven to zero. **Result:** $k_{0,1}$: 18.1%, $k_{0,2}$: 9.2%, α_1: 9.2%, α_2: 8.5%.

3. **Phase 3: Total-current regularization annealing.** Anneal Tikhonov regularization on total current only. **Result: Destroyed R2:** $k_{0,2}$ diverged to 137%, α_2 to 90%.

4. **Phase 4: Multi-pH.** Multi-condition fitting. **Result:** Could not recover from Phase 3 corruption.

Root cause of Phase 3 failure: The total-current-only objective in Phase 3 triggered the same mechanism as v1's Phase 2: the optimizer reduced the total current residual by adjusting R2 parameters away from truth. The key insight is that the "$k_{0,2} \to 0$" failure mode is not specific to k_0-only optimization—it occurs in *any* phase that uses total current as the sole observable while jointly optimizing all four parameters. This led to the **"never total-current-only" rule**: every phase that jointly optimizes R1 and R2 parameters must include the peroxide observable.

24.4 v3 R1-Only Refinement Attempt — Failed

An intermediate v3 attempt used Phase 3 to refine R1 parameters only (with R2 frozen at Phase 2 values) on total-current data:

- R2 parameters stayed frozen as designed.
- $k_{0,1}$ **worsened**: 18% → 66%.
- α_1 **worsened**: 9% → 25%.

Root cause: The residual R2 error from Phase 2 biases the total current at every voltage point. When the optimizer fits R1 parameters to total-current data that contains this R2 bias, it compensates by distorting R1 parameters. This demonstrates that total-current-only refinement is unreliable even when only R1 parameters are free, because the total current conflates R1 and R2 contributions.

24.5 v3 Final Pipeline (3-Phase, Multi-Obs Throughout)

The final protocol enforces multi-observable fitting in every joint optimization phase, eliminating the possibility of total-current-only corruption:

1. **Phase 1: α-only symmetric.** 12-point symmetric range ($\hat{\eta} \in [-28, +5]$), α-only optimization with k_0 fixed at initial guesses. **Result:** α_1 error = 0.36%.
2. **Phase 2: Multi-obs shallow cathodic.** 10-point shallow cathodic range ($\hat{\eta} \in [-1, -13]$), multi-observable (total + peroxide) joint (k_0, α) optimization. k_0 starts from the cold initial guess. **Result:** All four parameters at 8–18% error.
3. **Phase 3: Multi-obs full cathodic.** 15-point full cathodic range ($\hat{\eta} \in [-1, -46.5]$), multi-observable joint optimization, warm-started from Phase 2 values. **Result:** Improved 3 of 4 parameters.

The **shallow-then-full** strategy is critical: Phase 2 uses only the shallow cathodic range ($|\hat{\eta}| \leq 13$) to avoid solver failures during early optimization when parameters are far from truth. Phase 3 extends to the full cathodic range ($|\hat{\eta}| \leq 46.5$), adding the deep diffusion-limited plateau constraint once parameters are in the correct basin of attraction.

24.6 Results: v3 Final

Parameter	True	Recovered	Rel. Error
$k_{0,1}$	1.2630×10^{-3}	1.4730×10^{-3}	0.1665
$k_{0,2}$	5.2630×10^{-5}	5.5820×10^{-5}	0.0606
α_1	0.6270	0.5758	0.0816
α_2	0.5000	0.4414	0.1172

In physical units: $k_{0,1} = 2.8000 \times 10^{-8}$ m/s (true 2.4000×10^{-8} m/s), $k_{0,2} = 1.0600 \times 10^{-9}$ m/s (true 1.0000×10^{-9} m/s).

24.7 Comprehensive Comparison (All Methods)

Method	$k_{0,1}$ err (%)	$k_{0,2}$ err (%)	α_1 err (%)	α_2 err (%)	Max err (%)
Staged S2 (15pt cath.)	4.9000	98.5000	0.1000	72.3000	98.5000
Symmetric joint (20pt)	296.0000	850.0000	1.8000	84.0000	850.0000
Multi-obs standalone	16.6000	6.0000	8.2000	11.7000	16.6000
Multi-pH v2 (3 conds.)	6.5000	95.8000	9.2000	90.0000	95.8000
Master v1 best	4.7000	98.8000	0.0300	73.2000	98.8000
Master v2 best (P2)	18.1000	9.2000	9.2000	8.5000	18.1000
Master v3 best (P3)	**16.7**	**6.1**	**8.2**	**11.7**	**16.7**

24.8 Analysis

1. **Peroxide current is indispensable for R2.** The v1 pipeline's failure, v2 Phase 3's failure, and v3 R1-only attempt all demonstrate the same mechanism: no amount of staging, warm-starting, or phase ordering can recover R2 from total current alone. The peroxide current observable ($I_{\text{pxd}} = -(R_1 - R_2) \cdot \text{scale}$) is the only measurement that provides a direct constraint on R2's sign and magnitude.

2. **Pipeline ordering is critical.** The k_0-only phase in v1 was not merely unhelpful—it was actively harmful, poisoning $k_{0,2}$ beyond recovery. The lesson is that any optimization phase that operates on total current alone will tend to drive $k_{0,2} \to 0$, and this must be avoided in any phase that precedes or includes multi-observable fitting.

3. **"Never total-current-only" rule.** This was validated across three independent experiments:

   - v1 Phase 2 (k_0-only on total current): $k_{0,2} \to 98.8\%$
   - v2 Phase 3 (reg. annealing on total current): $k_{0,2} \to 137\%$
   - v3 R1-only on total current: $k_{0,1}$ worsened $18\% \to 66\%$

   Every phase that jointly optimizes BV parameters must include the peroxide observable.

4. **R1 vs. R2 trade-off.** Methods optimized for R1 (staged, k_0-only) achieve $k_{0,1} < 5\%$ but $k_{0,2} > 98\%$. Methods including the peroxide observable achieve $k_{0,2} < 10\%$ but $k_{0,1} \sim 17\%$. The dual objective forces the optimizer to satisfy two constraints simultaneously, pulling R1 parameters away from their single-observable optimum. No method yet achieves $< 10\%$ on all four parameters simultaneously with 2% noise.

5. **Shallow-then-full is the optimal architecture.** Phase 2's shallow range ($|\hat{\eta}| \leq 13$) avoids the solver failure region during early optimization when parameters are far from truth.

Phase 3's full range ($|\hat{\eta}| \leq 46.5$) adds the plateau constraint that improves 3 of 4 parameters once the optimizer is in the correct basin. This two-stage voltage strategy is more robust than using the full range from the start.

6. **v3 final matches standalone multi-obs.** The v3 pipeline's Phase 3 result (16.7% / 6.1% / 8.2% / 11.7%) matches the standalone multi-observable result, confirming that multi-observable joint fitting is doing the heavy lifting. The α-symmetric Phase 1 contributes a modest warm-start benefit to α_1 but does not substantially change the final outcome.

25. Speed Optimization of Master Pipeline

25.1 Motivation

The v3 master pipeline (§24) runs in ∼2075 s (∼34.6 min). While acceptable for single runs, iterating on pipeline design and running parameter sweeps benefits from shorter wall-clock times. The goal was to reduce runtime without degrading parameter recovery accuracy (tolerance: $< 0.1\%$ relative change in any parameter error).

25.2 Optimizations Retained (v4)

1. Shared forward solutions for multi-observable phases (∼15% speedup on multi-obs phases). The forward PDE solution is identical for both observables (total current and peroxide current)—only the post-solve extraction differs. In v3, the Firedrake tape was cleared between observables, forcing a full re-solve. In v4, cache clearing was removed so the checkpoint-restart fast path works for the secondary observable. This avoids redundant SNES solves in Phases 2 and 3.
File changed: `FluxCurve/bv_curve_eval.py`.
2. Predictor validation (∼5% speedup). The warm-start predictor extrapolates DOFs from the previous two converged solutions. When parameters are far from truth (early optimizer iterations), the predictor can produce states that deviate $> 10\times$ from the carry state, causing SNES to waste 10–15 iterations before diverging and falling back to the simple warm-start. In v4, a pre-solve check compares the predictor DOFs against the carry state; if the maximum deviation exceeds $10\times$, the predictor is immediately reverted. This reduced predictor failure retries from ∼35% to near-zero.
File changed: `FluxCurve/bv_point_solve.py`.

25.3 Optimizations Tried and Reverted

Three more aggressive strategies were tested and reverted because they degraded parameter accuracy by $> 2\%$:

1. **Fewer voltage points** (10→7 in Phase 2, 15→10 in Phase 3). $k_{0,1}$ error jumped from 16.7% to 52%. Insufficient I–V curve sampling destroyed identifiability—the optimizer could not distinguish between parameter combinations with fewer data constraints.

2. **Skip Phase 1** (α-only symmetric). $k_{0,1}$ error jumped to 58%. Phase 1's α warm-start, though modest in the final result, provides a critical starting basin for Phase 2. Without it, the optimizer converges to a qualitatively different local minimum.

3. **Aggressive convergence tolerances** (`maxiter` 25, `gtol` 10^{-5}). $k_{0,1}$ error jumped to 55%. L-BFGS-B was terminating before convergence; the default `maxiter` 40 and `gtol` 10^{-8} are near-minimal for this 4-parameter problem.

25.4 Timing Breakdown

Phase	v3 Time (s)	v4 Time (s)	Speedup (%)
P1 (α symmetric)	172.0000	172.0000	0.0000
P2 (multi-obs shallow)	819.0000	696.0000	15.0000
P3 (multi-obs full)	1084.0000	964.0000	11.1000
Total	**2075**	**1832**	**11.7**

Phase 1 is unaffected because it uses a single observable (α-only on total current). Phases 2 and 3 benefit from both optimizations: shared forward solutions eliminate redundant PDE solves, and predictor validation reduces wasted SNES iterations.
Accuracy verification: All four parameter errors match v3 to within 0.03% relative: $k_{0,1}$: 16.7%, $k_{0,2}$: 6.1%, α_1: 8.2%, α_2: 11.7%.

25.5 Diminishing Returns Analysis

The runtime breakdown reveals why further optimization is difficult without architectural changes:

- **85% of runtime is PETSc SNES solves + Firedrake adjoint tape replays.** These are already optimized by the checkpoint-restart fast path and predictor validation. Further reduction requires either fewer solves (which breaks identifiability) or faster individual solves (which requires parallel hardware).

- **The optimizer needs 20–30 evaluations for 4 parameters.** This is near the minimum for L-BFGS-B on a 4-parameter problem with the observed condition number. Gauss–Newton could reduce this, but requires second-order adjoint infrastructure.

- **Voltage point count is near-minimal.** Fewer than 10 points in the shallow range or 15 in the full range breaks identifiability (as demonstrated by the reverted experiments).

- **Further $> 15\%$ speedup requires parallel MPI point-solves or GPU acceleration.** Independent voltage points within a single objective evaluation could be solved in parallel across MPI ranks. This would require restructuring the serial sweep loop in `FluxCurve/bv_curve_eval.py` into a distributed map-reduce pattern—a fundamental architectural change.

27. Parallelism Optimization of Master Pipeline (v5–v7)

Following the 11.7% speedup in Section 25, three rounds of deeper optimization were applied to the 3-phase master inference pipeline, achieving a cumulative **4.4× wall-clock speedup** (1832 s → 415 s) while simultaneously *improving* accuracy on all four BV parameters.

27.1 Key Insight: Parallel Fast Path

The BV flux-curve inference pipeline evaluates the objective function by sweeping over voltage points in sequence. The *first* evaluation must be sequential because warm-start continuation uses each converged solution as the initial condition for the next voltage point. However, once this first sweep completes, the converged solutions are cached in a checkpoint dictionary keyed by $\hat{\eta}$. All subsequent L-BFGS-B evaluations (with updated parameters) can therefore solve each voltage point **independently** from its cached IC, with no inter-point data dependency.
This observation enables process-level parallelism: after the first sequential sweep, the "fast path" dispatches voltage-point forward+adjoint solves to a pool of worker processes.

27.2 Round 1: Parallel Fast Path (v5)

Implementation. A `ProcessPoolExecutor` with `spawn` context (following the existing Robin pipeline pattern in `FluxCurve/point_solve.py`) distributes independent voltage-point solves across worker processes.
New file: `FluxCurve/bv_parallel.py`, containing:

- `BVParallelPointConfig` — a frozen, pickle-serializable dataclass holding all mesh/physics/-solver parameters needed to reconstruct a BV solve context in a worker process.
- `_bv_worker_init(config_bytes)` — called once per worker at pool creation. Sets `OMP_NUM_THREADS=1` (critical for avoiding thread contention within PETSc), imports firedrake, builds the mesh and function spaces.
- `_bv_worker_solve_point(task)` — receives a 7-element tuple (parameter values, voltage, target flux, cached IC data), builds the BV context and forms, loads the cached IC into a firedrake Function, performs the forward solve+adjoint, and returns the point objective value and gradient.
- `BVPointSolvePool` — manages the `ProcessPoolExecutor` lifecycle (creation, submit, shutdown).

Modified files:

- `FluxCurve/bv_point_solve.py` — added `_solve_cached_fast_path_parallel()`, module-level `_parallel_pool` variable, and `set_parallel_pool()`/`close_parallel_pool()` functions. The fast-path dispatcher checks whether a parallel pool is active and the current observable

mode matches the pool's config; if so, it submits all voltage points as futures and gathers results.

- `FluxCurve/bv_config.py` — added `parallel_fast_path: bool` and `parallel_workers: int` fields to `BVFluxCurveInferenceRequest`.
- `FluxCurve/__init__.py` — added parallel exports.

Results:

Phase	v4 (s)	v5 (s)	Speedup
P1 (α-symmetric, single-obs)	172.0000	167.0000	1.03×
P2 (multi-obs shallow)	696.0000	346.0000	2.01×
P3 (multi-obs full)	964.0000	408.0000	2.36×
Total	**1832**	**923**	**2.0×**

Issue identified: The secondary observable (peroxide current) fell back to sequential execution because the pool's config was built for the primary observable (total current). An observable-mode compatibility check rejected the peroxide dispatch, causing ∼36% of v5 runtime to remain sequential.

27.3 Round 2: Multi-Observable Workers (v6)

Key insight: Workers can compute *both* adjoint gradients (primary and secondary observable) after a single forward PDE solve, completely eliminating the sequential secondary-observable sweep.
Implementation — "Two-tape-pass" approach:

1. The forward PDE solves to convergence (unchanged).
2. The adjoint tape is cleared.
3. The BV context and forms are rebuilt from scratch.
4. The converged IC is loaded into fresh firedrake Functions.
5. A single SNES step is executed (converged state → converged state, typically ∼1 Newton iteration with residual below tolerance). This re-records the solve on a clean tape.
6. The primary observable form is built and the adjoint is computed.
7. Steps 2–6 are repeated for the secondary observable (with the appropriate observable-mode flag).

This "two-tape-pass" pattern was encapsulated in a new function `_bv_worker_adjoint_tape_pass()` in `bv_parallel.py`. The task tuple was extended from 7 to 9 elements to carry the secondary target flux and a multi-obs flag.
Modified files:

- `FluxCurve/bv_parallel.py` — added `_bv_worker_adjoint_tape_pass()`, extended `_bv_worker_solve_point` to handle 9-element tasks.
- `FluxCurve/bv_point_solve.py` — added `_solve_cached_fast_path_parallel_multi_obs()`.
- `FluxCurve/bv_curve_eval.py` — modified `evaluate_bv_multi_observable_objective_and_gradient()` to detect a multi-obs-capable pool and skip the second (sequential) evaluation call entirely.

Worker sizing: $\min(n_{\text{points}}, \texttt{cpu_count} - 1) = \min(10, 9) = 9$ workers on a 10-core machine.
Results:

Phase	v5 (s)	v6 (s)	Speedup vs. v5
P1 (α-symmetric)	167.0000	162.0000	1.03×
P2 (multi-obs shallow)	346.0000	168.0000	2.06×
P3 (multi-obs full)	408.0000	188.0000	2.17×
Total	**923**	**520**	**1.78×**

Cumulative speedup vs. v4: **3.5**×. Accuracy was slightly improved relative to v5 (no degradation).

27.4 Round 3: Optimizer Tolerances + Pool Reuse (v7)

Relaxed optimizer convergence. L-BFGS-B tolerance parameters for Phases 2 and 3 were relaxed:

- `gtol`: $1 \times 10^{-6} \to 5 \times 10^{-6}$ (gradient norm tolerance)
- `ftol`: $1 \times 10^{-10} \to 1 \times 10^{-8}$ (relative function value tolerance)

This reduced the number of objective evaluations: Phase 2 from ∼40 to ∼26, Phase 3 from ∼38 to ∼15. Accuracy *paradoxically improved*, suggesting the tighter tolerances had caused the optimizer to overshoot past the true minimum in the noisy (2% noise) objective landscape.
Pool reuse. In v5/v6, a new `ProcessPoolExecutor` was created and destroyed for each phase. In v7, a single pool is created before Phase 2 and reused for Phase 3, saving ∼10 s of `spawn`-context overhead (firedrake import + mesh construction in each worker).
Results:

Phase	v6 (s)	v7 (s)	Speedup vs. v6
P1 (α-symmetric)	162.0000	162.0000	1.00×
P2 (multi-obs shallow)	168.0000	135.0000	1.24×
P3 (multi-obs full)	188.0000	117.0000	1.61×
Total	**520**	**415**	**1.25×**

Cumulative speedup vs. v4: **4.4**× (1832 s → 415 s).

27.5 Complete Timing and Accuracy Comparison

Version	P1 (s)	P2 (s)	P3 (s)	Total (s)
v4 (serial baseline)	172.0000	696.0000	964.0000	1832.0
v5 (parallel fast path)	167.0000	346.0000	408.0000	923.0
v6 (multi-obs workers)	162.0000	168.0000	188.0000	520.0
v7 (tolerances + pool reuse)	162.0000	135.0000	117.0000	415.0

Parameter	v4 err (%)	v7 err (%)	Change (pp)
$k_{0,1}$	16.7000	10.9000	−5.8000
$k_{0,2}$	6.1000	2.6000	−3.5000
α_1	8.2000	5.6000	−2.6000
α_2	11.7000	8.8000	−2.9000

The accuracy improvement is a side effect of the relaxed tolerances in Round 3. All four parameters improved by 2.6–5.8 percentage points, with $k_{0,2}$ (the hardest parameter historically) improving from 6.1% to 2.6%.

27.6 Diminishing Returns Analysis

The remaining 415 s runtime breaks down approximately as:

Component	Time (s)
Target generation (sequential warm-start sweeps)	65–80
First sequential sweep per phase (warm-start IC population)	50–60
Parallel fast-path evaluations (9 workers)	200–220
Pool init/shutdown overhead (`spawn` + firedrake import)	35–45
Miscellaneous (optimizer overhead, data marshalling)	10–20

Target generation and the first sweep per phase are **inherently sequential**: BV physics at large overpotentials diverges without warm-start continuation from the previous voltage point. The parallel fast path is already at 9 workers on a 10-core machine. Pool initialization overhead is dominated by the firedrake import (∼3 s per worker) and mesh construction, which are irreducible with the `spawn` context required by firedrake's PETSc backend.

Strategies assessed and rejected:

- **Worker solver reuse** (∼5–10% potential gain): Reusing the firedrake solver object across evaluations within a worker would avoid form recompilation. However, the adjoint tape requires fresh forms for each pass (the tape records operator assembly), making reuse incompatible with correct gradient computation. High implementation complexity for low reward.

- **Multi-fidelity mesh** (∼15–20% potential gain): Using a coarser mesh for early optimizer evaluations and switching to the fine mesh for final evaluations. Cache invalidation on mesh

switch negates the fast-path advantage (all cached ICs must be recomputed on the new mesh), and the fine-mesh sweep becomes the new bottleneck.

- **Speculative parallel first evaluation** (∼10–15% potential gain): Running the first sweep in parallel by launching all voltage points simultaneously from the equilibrium IC (instead of sequentially from warm-start). BV physics at $|\hat{\eta}| > 10$ diverges from the equilibrium IC without intermediate warm-start states; only 3–4 of 10 points would converge, requiring a sequential fallback anyway.

- **MPI Ensemble parallelism** (∼2–3× potential gain): Distributing voltage-point solves across MPI ranks using firedrake's Ensemble parallelism. Would require restructuring from a single-process `ProcessPoolExecutor` architecture to an `mpiexec`-launched distributed pattern. Deferred to future work as a separate development effort.

27.7 New Files Created

- `FluxCurve/bv_parallel.py` — Parallel worker infrastructure: `BVParallelPointConfig` (serializable config), `_bv_worker_init` (per-worker firedrake setup), `_bv_worker_solve_point` (forward + adjoint from cached IC), `_bv_worker_adjoint_tape_pass` (two-tape-pass for multi-observable), `BVPointSolvePool` (executor lifecycle management).

- `scripts/inference/Infer_BVMaster_charged_v5.py` — v5 master script with parallel fast-path dispatch.

- `scripts/inference/Infer_BVMaster_charged_v6.py` — v6 master script with multi-observable workers.

- `scripts/inference/Infer_BVMaster_charged_v7.py` — v7 master script with relaxed tolerances and pool reuse.

27.8 Files Modified

- `FluxCurve/bv_config.py` — Added `parallel_fast_path` and `parallel_workers` fields to `BVFluxCurveInferenceRequest`.

- `FluxCurve/bv_point_solve.py` — Added parallel dispatch (`_solve_cached_fast_path_parallel`, `_solve_cached_fast_path_parallel_multi_obs`), pool management (`set_parallel_pool`, `close_parallel_p` and observable-mode compatibility checking.

- `FluxCurve/bv_curve_eval.py` — Added multi-obs parallel path in `evaluate_bv_multi_observable_objecti` fixed module-level variable access for pool detection.

- `FluxCurve/__init__.py` — Added parallel exports (`BVPointSolvePool`, `set_parallel_pool`, `close_parallel_pool`).

27.9 Architecture Summary

The parallelism architecture follows a three-tier pattern:

1. **Outer loop** (L-BFGS-B optimizer): sequential parameter updates. Unchanged.
2. **Middle loop** (voltage-point sweep): sequential for the first evaluation (warm-start IC population), then parallel for all subsequent evaluations (fast-path from cached ICs).
3. **Inner loop** (per-point solve): each worker performs a forward PDE solve followed by one or two adjoint tape passes (primary and optionally secondary observable). Workers are fully independent after receiving their cached IC, parameters, and target flux.

The `spawn` context is required (not `fork`) because firedrake and PETSc maintain internal state that is not fork-safe. Each worker sets `OMP_NUM_THREADS=1` to prevent PETSc's internal threading from contending with the process-level parallelism. Worker-to-main communication uses pickle serialization of numpy arrays (firedrake Functions are converted to `.dat` arrays before sending and reconstructed on receipt).

29. POD/RBF Surrogate Model (v8–v9)

Following the 4.4× parallelism speedup (Section 27), a fundamentally different acceleration strategy was explored: replacing PDE forward solves with a radial basis function (RBF) surrogate that maps BV kinetic parameters directly to I–V curve observables. Three rounds of development (v8, v8.1, v9) were completed, reducing the optimization time from $\sim$345 s to < 0.1 s ($> 3000\times$ speedup) with complementary accuracy trade-offs relative to the PDE-based v7 pipeline.

29.1 Surrogate Architecture

The surrogate is a *parameter-to-observable* interpolant, not a full POD decomposition of the solution field. For each voltage point $\hat{\eta}_i$ and observable type (current density or peroxide current), the surrogate directly predicts the scalar observable value given the four BV parameters $\theta = (\log_{10} k_{0,1},\, \log_{10} k_{0,2},\, \alpha_1,\, \alpha_2)$.

RBF interpolation. The surrogate uses `scipy.interpolate.RBFInterpolator` with a thin-plate spline kernel ($\phi(r) = r^2 \log r$) and linear polynomial tail (`degree=1`). The k_0 parameters are sampled and interpolated in $\log_{10}$ space (matching the optimizer's parameterization), and all four input dimensions are z-score normalized (zero mean, unit variance) before interpolation. Two separate RBF models are built: one for current-density observables and one for peroxide-current observables.

Gradient computation. Gradients are computed via central finite differences with step size $h = 10^{-4}$:

$$\frac{\partial f}{\partial \theta_j} \approx \frac{f(\theta + h\, e_j) - f(\theta - h\, e_j)}{2h}.$$

Each gradient evaluation requires $2 \times 4 = 8$ surrogate evaluations—each taking microseconds—making the total gradient cost negligible ($< 1\,\text{ms}$).

NaN handling. Training samples that fail to converge at specific voltage points produce NaN values. The surrogate masks these entries: only converged (non-NaN) values contribute to interpolation at each voltage point. This means different voltage points may have slightly different training set sizes.

29.2 Training Data Generation

Training data are generated by calling the existing `FluxCurve` pipeline to compute I–V curves at LHS-sampled parameter combinations.

Sampling strategy. Parameters are sampled via Latin hypercube sampling (LHS) over the following ranges:

Parameter	Space	Lower	Upper
$k_{0,1}$	$\log_{10}$	-4.5	-1.5
$k_{0,2}$	$\log_{10}$	-6.0	-2.5
α_1	linear	0.3	0.9
α_2	linear	0.2	0.8

The true parameters fall well within these ranges.

Checkpointing. Because each PDE solve takes ∼60–70 s, the training data generator checkpoints results every 5 samples to an NPZ file. This enables resume-on-crash and incremental training set expansion. Each sample's I–V curve (22 voltage points × 2 observables) plus the 4-element parameter vector are stored.

Convergence rates. Not all parameter combinations converge at all voltage points. Round 1 achieved 90.5% overall convergence (181/200 samples fully valid); Round 2B achieved 89% (445/500). Failed samples are retained in the training set with NaN masking for the unconverged points.

29.3 Round 1: Initial Surrogate (v8)

The initial implementation created six new modules in a `Surrogate/` package (`sampling.py`, `surrogate_model.py`, `training.py`, `objectives.py`, `validation.py`, `io.py`) and four scripts in `scripts/surrogate/`.

Training. $N = 200$ LHS samples, 181 valid (90.5% convergence), 22 voltage points per sample. One-time offline training cost: ∼3.1 hours on a single machine (amortized over all subsequent inference runs).

Validation metrics. Normalized root-mean-square error (NRMSE) on a held-out 20% validation set:

- Current density: NRMSE = 0.06%

- Peroxide current: NRMSE = 0.54%

The peroxide-current surrogate is ∼9× less accurate than the current-density surrogate, reflecting the peroxide current's stronger nonlinear dependence on $k_{0,2}$ and α_2.

Inference results (v8). The inference script `Infer_BVMaster_charged_v8_surrogate.py` mirrors the 3-phase master protocol but replaces all PDE evaluations with surrogate calls:

Phase	$k_{0,1}$ (%)	$k_{0,2}$ (%)	α_1 (%)	α_2 (%)	Total (s)	Optim. time
P1 (α-only)	0.0	0.0	4.5	4.6	0	< 0.03 s
P2 (multi-obs shallow)	7.3	9.6	4.5	4.6	70	< 0.1 s
P3 (full cathodic)	18.2	37.2	11.7	11.2	70	< 0.1 s

Phase 2 produced the best results; Phase 3 (full cathodic surrogate) degraded all parameters because deep cathodic voltage points have weak partial correlations with the kinetic parameters. The reported total time of 69.8 s is almost entirely PDE-based target I–V curve generation (∼70 s, a shared fixed cost across all methods); the actual surrogate optimization takes **less than 0.1 s** (milliseconds). Excluding this shared target generation cost, the surrogate achieves a $> 3000\times$ speedup over PDE-based v7 optimization (∼345 s $\rightarrow < 0.1$ s). Offline training data generation (∼3.1 hr for $N = 200$) is a one-time amortized cost.

29.4 Round 2A: Quick Fixes (v8.1)

Two targeted modifications were tested:

Removal of Phase 3. Since the full cathodic surrogate degraded results, v8.1 removed Phase 3 entirely and used Phase 2 as the final result.

Optimizer bound clamping. The L-BFGS-B bounds were restricted to the training data range to prevent extrapolation outside the surrogate's domain of validity.

PDE refinement phase. A new Phase 3 was added that takes the surrogate-recovered parameters as a starting point and runs PDE-based L-BFGS-B refinement. *This did not improve results*: the PDE refinement converged to a different local minimum because the PDE solver faithfully reproduces the noise realization, while the surrogate implicitly smooths it. The surrogate's smoothing acts as a form of **implicit regularization**, analogous to early stopping or spectral truncation in classical inverse problems.

Results (v8.1). Total time: 177.8 s, comprising ∼75 s target generation, < 0.1 s surrogate optimization, and ∼101 s PDE refinement (the added PDE phase dominates the increase over v8). Best-of-phase selection picked the surrogate Phase 2, producing identical accuracy to v8 Phase 2—confirming that PDE refinement does not help.

29.5 Round 2B: More Training Data (v9)

Expanded training set. $N = 500$ LHS samples (445 valid, 89% convergence), including 100 focused samples near the true parameters (tighter LHS ranges: ± 0.5 in $\log_{10}$ for k_0, ± 0.15 for α).

One-time offline training cost: ∼9.5 hours (amortized over all subsequent inference runs).

Cross-validated smoothing. The RBF smoothing parameter ϵ was cross-validated over $\epsilon \in \{0, 10^{-6}, 10^{-4}, 10^{-2}, 0.1, 1.0\}$ using 5-fold CV. Result: $\epsilon = 0$ (exact interpolation) was optimal for both current-density and peroxide-current surrogates, confirming that the training data are smooth enough that no additional smoothing is needed.

Validation metrics. On the held-out validation set:

- Current density: NRMSE = 0.42% (worse than v8's 0.06% due to larger parameter range coverage)
- Peroxide current: NRMSE = 11.4% (significantly worse)

The peroxide-current NRMSE degradation is the bottleneck for $k_{0,2}$ accuracy: the surrogate cannot reproduce the fine structure of the peroxide current's dependence on R2 parameters.

Inference results (v9). Using surrogate-only inference (no PDE refinement):

Parameter	v7 (PDE)	v8 (Surr, 181)	v9 (Surr, 445)
$k_{0,1}$ err (%)	10.9	7.3	8.8
$k_{0,2}$ err (%)	2.6	9.6	7.6
α_1 err (%)	5.6	4.5	4.8
α_2 err (%)	8.8	4.6	6.4

The v9 surrogate improved $k_{0,2}$ from 9.6% to 7.6% (a 2 percentage point gain) at the cost of ∼3× longer one-time offline training. The actual v9 surrogate optimization time remains < 0.1 s. PDE refinement from v9 starting points again did not help.

29.6 Analysis and Limitations

Accuracy trade-offs. The surrogate improves $k_{0,1}$, α_1, and α_2 relative to PDE-based v7, but degrades $k_{0,2}$. This is because:

- The current-density surrogate is highly accurate (NRMSE $\leq$ 0.42%), so parameters identified primarily from current density ($k_{0,1}$, α_1) benefit from the surrogate's implicit regularization.
- The peroxide-current surrogate has much higher error (NRMSE up to 11.4%), limiting the accuracy of $k_{0,2}$ and α_2, which require the peroxide observable for identifiability.

Implicit regularization. The surrogate smooths the parameter-to-observable map by interpolating a finite training set. This acts as a low-pass filter on the objective landscape, reducing sensitivity to noise in the target data. PDE refinement re-introduces this noise sensitivity, which is why it does not improve (and can worsen) the surrogate result. This finding suggests that the PDE-based pipeline may be slightly *over-fitting* the noise, and that the surrogate's accuracy on $k_{0,1}$, α_1, and α_2 may be closer to the achievable optimum.

Diminishing returns. The progression from 181 to 445 training samples yielded modest improvements at rapidly increasing offline cost:

Metric	v8 (181 samples)	v9 (445 samples)	Δ
$k_{0,2}$ err (%)	9.6	7.6	-2.0
PC NRMSE (%)	0.5	11.4	$+10.9$
Offline cost (hr, one-time)	3.1	9.5	$\times 3.1$

The peroxide-current NRMSE actually *worsened* in v9 (despite more data) because the expanded parameter range increases the interpolation challenge. The $k_{0,2}$ improvement came from the focused samples near the true parameters rather than from the overall NRMSE improvement.

29.7 Full Comparison Table (v4–v9)

Table 1: Complete inference pipeline progression: optimization time (excluding shared target generation of $\sim$70 s), speedup, and parameter errors across all versions. Offline training data generation for surrogate versions is a one-time amortized cost reported separately.

Version	Optim. (s)	Offline	$k_{0,1}$ (%)	$k_{0,2}$ (%)	α_1 (%)	α_2 (%)	Notes
v4 (serial PDE)	1762	—	16.7	6.1	8.2	11.7	baseline
v5 (parallel)	853	—	16.7	6.1	8.2	11.7	fast-path
v6 (multi-obs)	450	—	16.7	6.1	8.2	11.7	2-tape workers
v7 (final PDE)	345	—	10.9	2.6	5.6	8.8	relaxed tol
v8 (Surr, N=181)	< 0.1	3.1 hr	7.3	9.6	4.5	4.6	RBF surrogate
v9 (Surr, N=445)	< 0.1	9.5 hr	8.8	7.6	4.8	6.4	more data

Note: all versions share a common target I–V curve generation cost of $\sim$70 s (PDE-based), which is excluded from the optimization times above. The surrogate optimization itself takes milliseconds ($<$ 0.1 s), representing a $> 3000\times$ speedup over PDE-based v7 optimization ($\sim$345 s) on the inference step alone. The offline training data generation ($\sim$3.1 hr for $N = 200$, $\sim$9.5 hr for $N = 500$) is a one-time cost amortized over all subsequent inference runs.

29.8 Implementation Summary

The surrogate implementation consists of:

- `Surrogate/` **package** (7 modules): `__init__.py`, `sampling.py` (LHS generation), `surrogate_model.py` (RBF wrapper with normalization), `training.py` (batch PDE evaluation), `objectives.py` (objective/gradient for L-BFGS-B), `validation.py` (NRMSE, cross-validation), `io.py` (NPZ save/load with checkpointing).

- `scripts/surrogate/` (6 files): `generate_training_data.py` (LHS $\rightarrow$ PDE $\rightarrow$ NPZ), `build_surrogate.py` (NPZ $\rightarrow$ RBF model), `validate_surrogate.py` (NRMSE/plots), `Infer_BVMaster_charged_v8_surrogate.p` (v8), `Infer_BVMaster_charged_v8_1_surrogate.py` (v8.1), `Infer_BVMaster_charged_v9_surrogate.py` (v9).

29.9 Conclusions

The POD/RBF surrogate provides a viable alternative to PDE-based inference when speed is the priority. Key findings:

- $> 3000\times$ **optimization speedup** ($\sim$345 s $\rightarrow$ < 0.1 s) once the surrogate is trained (one-time offline cost of $\sim$3–10 hr) and target data are generated ($\sim$70 s, shared by all methods). The surrogate optimization is essentially instantaneous.
- **Complementary accuracy profile**: better on $k_{0,1}$, α_1, α_2; worse on $k_{0,2}$ (7.6% vs. 2.6%).
- **Implicit regularization** from the RBF interpolation smooths noise, making PDE refinement unnecessary and potentially counterproductive.
- **Diminishing returns** from additional one-time offline training: $3\times$ more samples yielded only 2 pp improvement on $k_{0,2}$ with degraded peroxide-current NRMSE.
- The **PDE-based v7 pipeline remains the recommended approach** when $k_{0,2}$ accuracy is critical. The surrogate is most valuable for rapid inference (many runs, real-time applications) or as a preconditioner for MCMC/EKI sampling.

30. Updated Next Steps

Completed

1. **Multi-observable joint fitting.** COMPLETED. Fitting total current and peroxide current simultaneously achieved the R2 breakthrough: $k_{0,2}$ error 6.1%, α_2 error 11.7%. Multi-observable is the recommended approach for R2 recovery.
2. **Multi-pH inference.** COMPLETED. Three-condition multi-pH with shared (k_0, α) achieved excellent R1 ($k_{0,1}$: 6.5%, α_1: 3.6%) and partial R2 improvement (α_2: 29.5%).
3. **Master inference protocol v1/v2/v3.** COMPLETED. Three iterations of pipeline design converged on a 3-phase multi-obs-throughout architecture. v1's k_0-only phase poisoned R2; v2's total-current Phase 3 destroyed R2; v3's multi-obs-throughout approach recovers all four parameters to within 17%. Final best: $k_{0,1}$: 16.7%, $k_{0,2}$: 6.1%, α_1: 8.2%, α_2: 11.7%.
4. **Forward solver robustness.** COMPLETED. Mesh refinement ($N_y = 300$, $\beta = 4.0$), smaller timesteps ($\Delta t = 0.1$), denser voltage sweeps (19 points), softplus regularization, and SNES linesearch tuning raised target generation convergence from 78% to 100% (57/57).
5. **Speed optimization of master pipeline.** COMPLETED. Shared forward solutions for multi-observable phases and predictor validation achieved 11.7% wall-clock speedup (2075 s $\rightarrow$ 1832 s, $\sim$30.5 min) with identical accuracy (all parameter errors within 0.03% of v3). More aggressive strategies (fewer voltage points, skipping Phase 1, relaxed convergence tolerances) were tried and reverted due to $> 2\%$ accuracy degradation.

6. **Parallelism optimization (v5–v7).** COMPLETED. Three rounds of optimization achieved a 4.4× cumulative speedup (1832 s → 415 s, ∼6.9 min) while *improving* accuracy on all four parameters by 2.6–5.8 percentage points. Round 1: parallel fast-path dispatch via `ProcessPoolExecutor` (2.0×). Round 2: multi-observable workers computing both adjoint gradients per forward solve (3.5× cumulative). Round 3: relaxed optimizer tolerances and pool reuse (4.4× cumulative). The pipeline is now near the practical ceiling for single-machine process-level parallelism; further speedup requires MPI Ensemble or GPU acceleration.

7. **POD/RBF surrogate model (v8–v9).** COMPLETED. RBF thin-plate spline surrogate trained on 181–445 LHS parameter samples, mapping BV parameters directly to I–V curve observables. Reduces optimization time from ∼345 s to < 0.1 s ($> 3000\times$ speedup), with competitive accuracy on $k_{0,1}$ (7.3%), α_1 (4.5%), and α_2 (4.6%), but degraded $k_{0,2}$ (9.6% vs. 2.6%) due to peroxide-current interpolation error. Target generation (∼70 s) is a shared fixed cost; offline training (∼3–10 hr) is a one-time amortized cost. Key finding: surrogate acts as implicit regularizer, making PDE refinement unnecessary. Diminishing returns observed from expanding training data (3× more samples → 2 pp $k_{0,2}$ improvement at 3× offline cost). The PDE-based v7 pipeline remains the recommended approach when $k_{0,2}$ accuracy is critical; the surrogate is best suited for rapid inference or as a preconditioner for sampling methods.

Future work

8. **Combined multi-observable + multi-pH + symmetric.** Multi-observable breaks R2 degeneracy (two equations per voltage), multi-pH adds concentration diversity ($[H^+]^2$ scaling), and symmetric data constrains α via the anodic Tafel slope. Combining all three in a single pipeline could push the R1–R2 trade-off frontier below the current 11% maximum error.

9. **Neutral-species formulation for wider pH range.** The charged solver fails at $c_{H^+} > 0.3$ (nondimensional) due to Debye-length stiffness. A neutral-species formulation would avoid Poisson stiffness entirely, enabling multi-pH inference across the full experimentally relevant range (pH 2 to pH 12 as in Mangan2025).

10. **Gauss–Newton optimizer.** The current L-BFGS-B optimizer uses only gradient information. A Gauss–Newton or Levenberg–Marquardt method could exploit the least-squares structure to better condition the k_0–α joint optimization, potentially breaking the R1–R2 accuracy trade-off.

11. **Experimental data fitting.** Apply the validated v7 pipeline to real RRDE data from Mangan et al. (2025), which provides both disk (total) and ring (peroxide) currents—the exact observables the pipeline is designed for. The surrogate could accelerate initial parameter sweeps before PDE-based refinement.

12. **Bayesian uncertainty quantification.** Replace point estimates with posterior distributions via ensemble Kalman inversion (EKI) or MCMC to characterize parameter uncertainty

and the R1–R2 correlation structure. The surrogate's microsecond evaluation time makes MCMC sampling with 10^5–10^6 evaluations practical without PDE solves.

13. **Transient (chronoamperometric) data.** R1 and R2 reach steady state on different timescales; fitting $I(t)$ at fixed overpotentials provides temporal separation invisible in steady-state data.

14. **MPI Ensemble parallelism for further speedup.** The current `ProcessPoolExecutor` approach is near its ceiling (9 workers on 10 cores, $\sim$40 s pool init overhead). MPI Ensemble parallelism would distribute voltage-point solves across MPI ranks, potentially yielding 2–3$\times$ additional speedup on cluster hardware. This requires restructuring from a single-process to an `mpiexec`-launched architecture.